

NPTEL Course on "Computer System Optimization for ML"

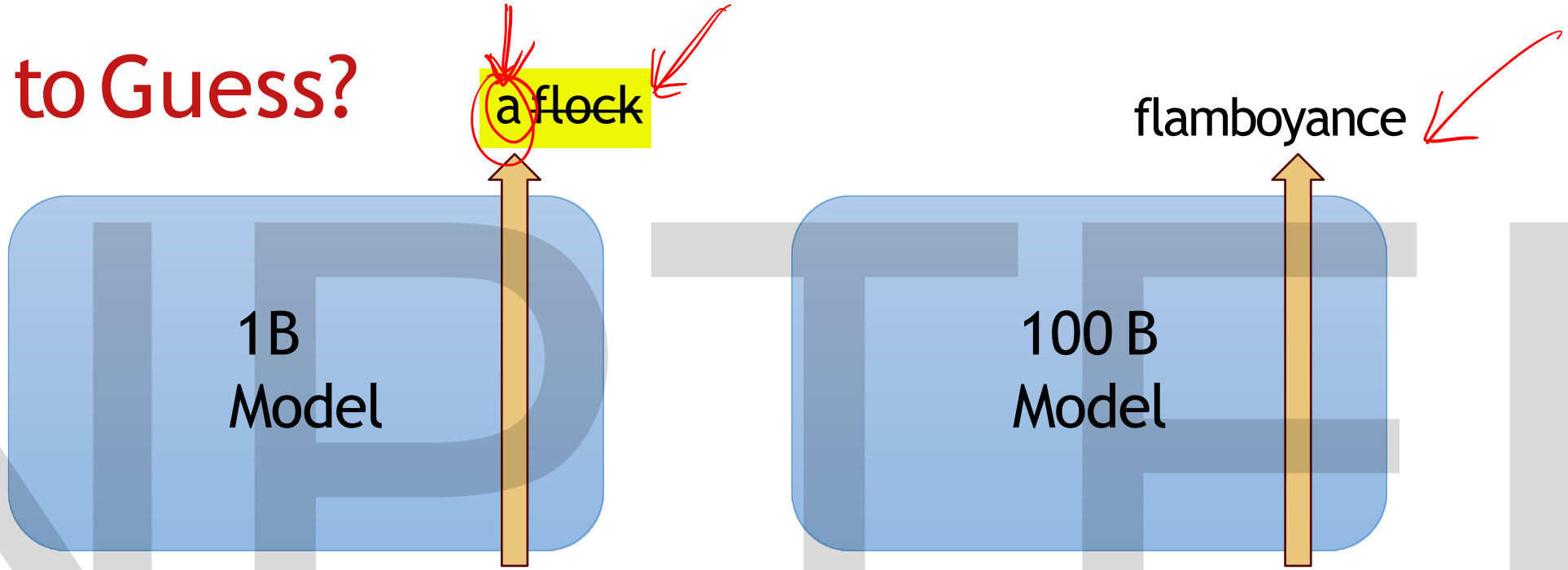
Lecture #27

Agenda

- Speculative Decoding (contd.)
- KV-caching

NPTEL

How to Guess?



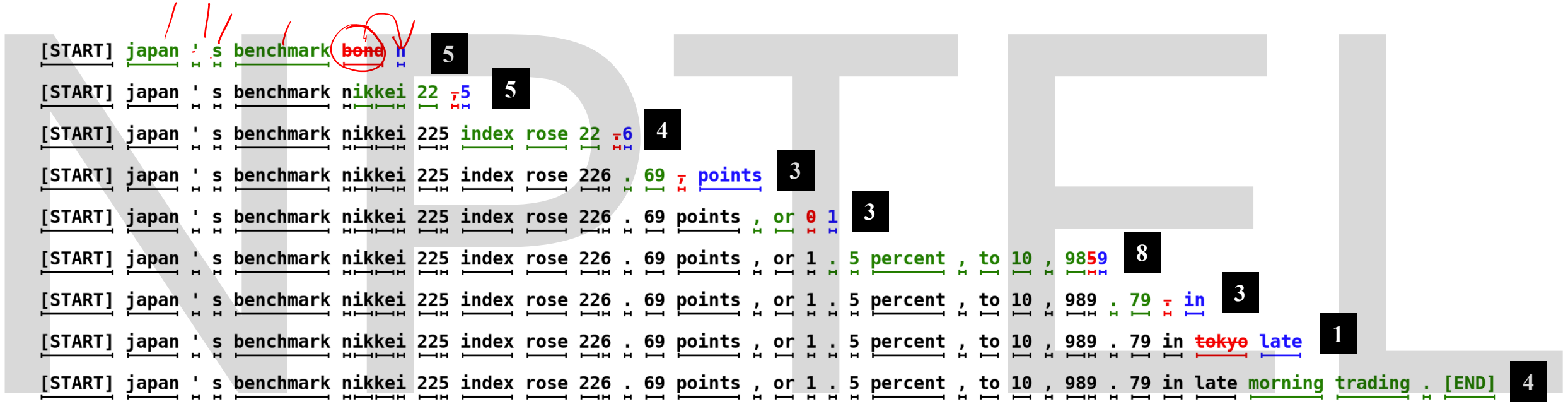
Very easy

Can use a small "*draft*" model to guess!

Difficult

Verify correct it using the "*target* model"

Example



37 forward pass vs 9 forward pass!

What about the forward passes through the draft model?

A hyperparamter

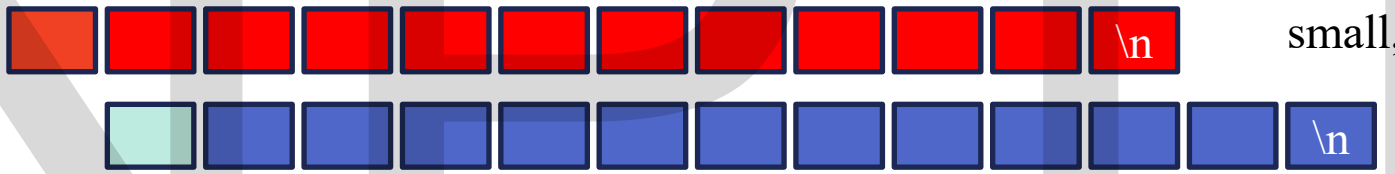
- How many tokens should we generate on the draft model before verifying it using the base model?



NPTEL

A hyperparamter

- **How many tokens should we generate on the draft model before verifying it using the base model?**

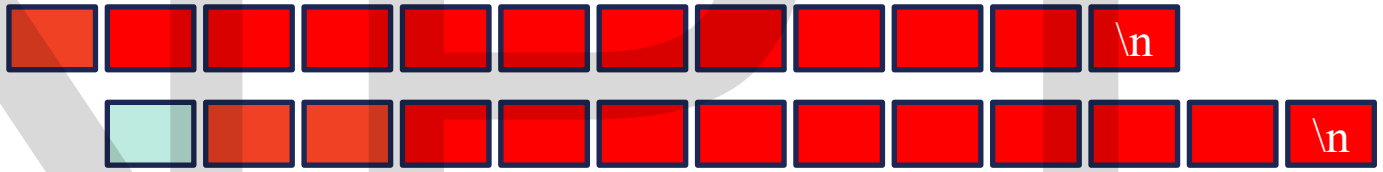


Wasted compute on the draft model Although small, it is still wasted GPU time

NPTEL

A hyperparamter

- **How many tokens should we generate on the draft model before verifying it using the base model?**



Wasted compute on the draft model Although small, it is still wasted GPU time

C = compute ratio of draft:base model => low c, high guess tokens per round

Speculative Sampling

- Greedy decoding
  - Picks the next token with max probability
  - Deterministic

- But what about sampling by varying – top-p, top-k, or temperature?
  - Sample the next decode token not deterministically pick the one with max prob.
  - Needed for variance, creativity, diversity

NPTEL

Speculative Sampling

NPTEL

DeepMind

2023-2-3

Accelerating Large Language Model Decoding with Speculative Sampling

Charlie Chen[1], Sebastian Borgeaud[1], Geoffrey Irving[1], Jean-Baptiste Lespiau[1], Laurent Sifre[1] and John Jumper[1]
[1]All authors from DeepMind

We present speculative sampling, an algorithm for accelerating transformer decoding by enabling the generation of multiple tokens from each transformer call. Our algorithm relies on the observation that the latency of parallel scoring of short continuations, generated by a faster but less powerful draft model, is comparable to that of sampling a single token from the larger target model. This is combined with a novel modified rejection sampling scheme which preserves the distribution of the target model within hardware numerics. We benchmark speculative sampling with Chinchilla, a 70 billion parameter language model, achieving a 2–2.5× decoding speedup in a distributed setup, without compromising the sample quality or making modifications to the model itself.

Fast Inference from Transformers via Speculative Decoding

Yaniv Leviathan [*1] Matan Kalman [*1] Yossi Matias [1]

Abstract

Inference from large autoregressive models like Transformers is slow - decoding K tokens takes K serial runs of the model. In this work we introduce *speculative decoding* - an algorithm to sample from autoregressive models faster *without any changes to the outputs*, by computing several tokens in parallel. At the heart of our approach lie the observations that (1) hard language-modeling tasks often include easier subtasks that can be approximated well by more efficient models, and developed to make inference from them faster. Some approaches aim to reduce the inference cost for *all* inputs equally (e.g. Hinton et al., 2015; Jaszczur et al., 2021; Hubara et al., 2016; So et al., 2021; Shazeer, 2019). Other approaches stem from the observation that not all inference steps are born alike - some require a very large model, while others can be approximated well by more efficient models. These *adaptive computation* methods (e.g. Har et al., 2021; Sukhbaatar et al., 2019; Schuster et al., 2021; Scardapane et al., 2020; Bapna et al., 2020; Elbayad et al., 2019; Schwartz et al., 2020) aim to use less compute re-

Content credits: https://youtu.be/S-8yr_RibJ4?si=Kv8xyyTsJvu8oKLV

Google Research

Algorithm

M_p = draft model

M_q = target model

∞ meta-llama/Llama-2-7b-chat-hf

∞ meta-llama/Llama-2-70b-chat-hf

pf = prefix, $K = 5$ tokens

NPTEL

Content credits: https://youtu.be/S-8yr_RibJ4?si=Kv8xyyTsJvu8oKLV

M_p = draft model

M_q = target model

∞ meta-llama / **Llama-2-7b-chat-hf**

∞ meta-llama / **Llama-2-70b-chat-hf**

Algorithm

pf = prefix, K = 5 tokens

$p_1(x) = M_p(pf)$ —Sample→ x_1

NPTEL

Content credits: https://youtu.be/S-8yr_RibJ4?si=Kv8xyyTsJvu8oKLV

Algorithm

M_p = draft model

M_q = target model

∞ meta-llama / **Llama-2-7b-chat-hf**

∞ meta-llama / **Llama-2-70b-chat-hf**

pf = prefix, $K = 5$ tokens

$p_1(x) = M_p(pf) \rightarrow x_1$

$p_2(x) = M_p(pf, x_1) \rightarrow x_2$

...

$p_5(x) = M_p(pf, x_1, x_2, x_3, x_4) \rightarrow x_5$

NPTEL

Content credits: https://youtu.be/S-8yr_RibJ4?si=Kv8xyyTsJvu8oKLV

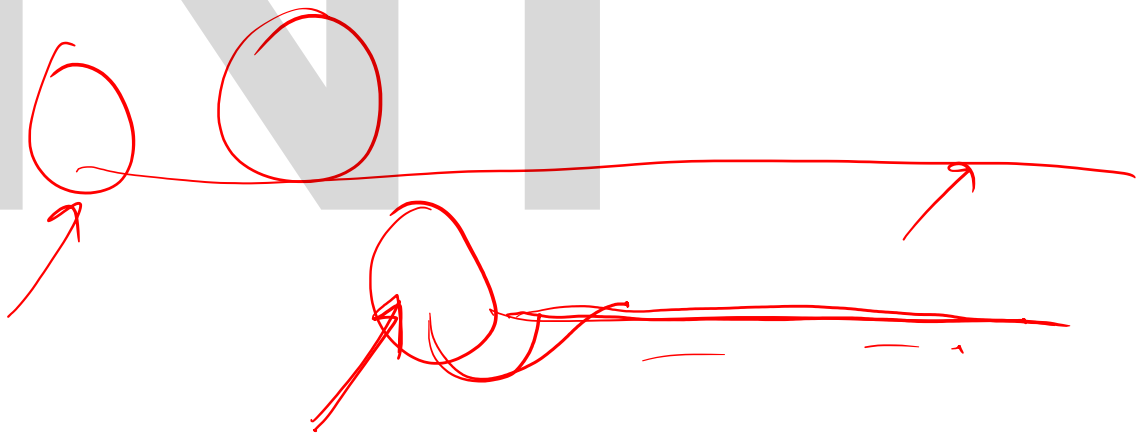
$$p_1(x) = M_p(pf) \longrightarrow x_1$$

$$p_2(x) = M_p(pf, x_1) \longrightarrow x_2$$

...

$$p_5(x) = M_p(pf, x_1, x_2, x_3, x_4) \longrightarrow x_5$$

Run draft model for K steps



NPTEL

Content credits: https://youtu.be/S-8yr_RibJ4?si=Kv8xyyTsJvu8oKLV

$$p_1(x) = M_p(pf) \longrightarrow x_1$$

$$p_2(x) = M_p(pf, x_1) \longrightarrow x_2$$

...

$$p_5(x) = M_p(pf, x_1, x_2, x_3, x_4) \longrightarrow x_5$$

Run draft model for K steps

A distribution at each step over entire vocabulary

$$q_1(x), q_2(x), q_3(x), q_4(x), q_5(x), q_6(x)$$

$$= M_q(pf, x_1, x_2, x_3, x_4, x_5)$$

Run target model once

NPTEL

Content credits: https://youtu.be/S-8yr_RibJ4?si=Kv8xyyTsJvu8oKLV

$$p_1(x) = M_p(pf) \longrightarrow x_1$$

$$p_2(x) = M_p(pf, x_1) \longrightarrow x_2$$

...

$$p_5(x) = M_p(pf, x_1, x_2, x_3, x_4) \longrightarrow x_5$$

Draft Model

Target Model

$$q_1(x), q_2(x), q_3(x), q_4(x), q_5(x), q_6(x)$$

$$= M_q(pf, x_1, x_2, x_3, x_4, x_5)$$

Token	x1	x2	x3	x4	x5
	dogs	love	chasing	after	cars
p(x)	0.8	0.7	0.9	0.8	0.7
q(x)	0.9	0.8	0.8	0.3	0.8

Content credits: https://youtu.be/S-8yr_RibJ4?si=Kv8xyyTsJvu8oKLV

Rejection Sampling

	Token	x1	x2	x3	x4	x5
		dogs	love	chasing	after	cars
Draft Model	p(x)	0.8	0.7	0.9	0.8	0.7
Target Model	q(x)	0.9	0.8	0.8	0.3	0.8

NPTEL

Rejection Sampling

Token	x1	x2	x3	x4	x5
	dogs	love	chasing	after	cars
Draft Model p(x)	0.8	0.7	0.9	0.8	0.7
Target Model q(x)	0.9	0.8	0.8	0.3	0.8

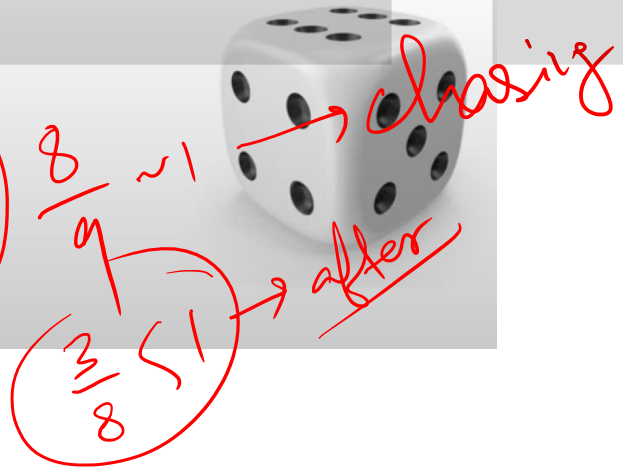
Case 1: If $q(x) \geq p(x)$, then accept

Rejection Sampling

Token	x1	x2	x3	x4	x5
	dogs	love	chasing	after	cars
Draft Model p(x)	0.8	0.7	0.9	0.8	0.7
Target Model q(x)	0.9	0.8	0.8	0.3	0.8

Case 1: If $q(x) \geq p(x)$, then accept

Case 2: If $q(x) < p(x)$, then accept with probability $\frac{q(x)}{p(x)}$



Rejection Sampling

Token	x1	x2	x3	x4	x5
	dogs	love	chasing	after	cars
Draft Model p(x)	0.8	0.7	0.9	0.8	0.7
Target Model q(x)	0.9	0.8	0.8	0.3	0.8

Case 1: If $q(x) \geq p(x)$, then accept

Case 2: If $q(x) < p(x)$, then accept with probability $\frac{q(x)}{p(x)}$



$$p_1(x) = M_p(pf) \longrightarrow x_1$$

$$p_2(x) = M_p(pf, x_1) \longrightarrow x_2$$

...

$$p_5(x) = M_p(pf, x_1, x_2, x_3, x_4) \longrightarrow x_5$$

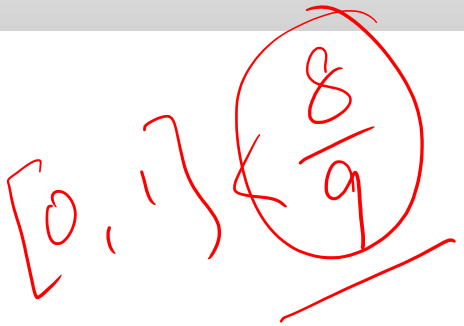
Draft Model

Target Model

$$q_1(x), q_2(x), q_3(x), q_4(x), q_5(x), q_6(x)$$

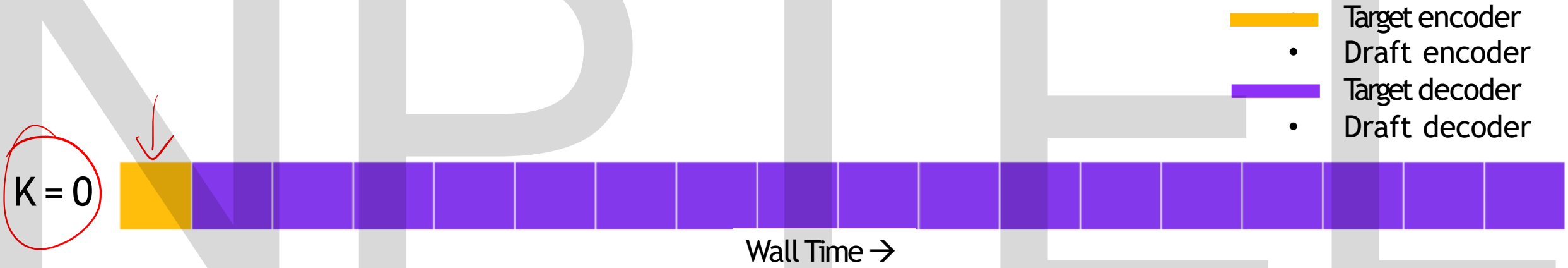
$$= M_q(pf, x_1, x_2, x_3, x_4, x_5)$$

Token	x1	x2	x3	x4	x5
	dogs	love	chasing	after	cars
p(x)	0.8	0.7	0.9	0.8	0.7
q(x)	0.9	0.8	0.8	0.3	0.8
	✓	✓	✓	✗	



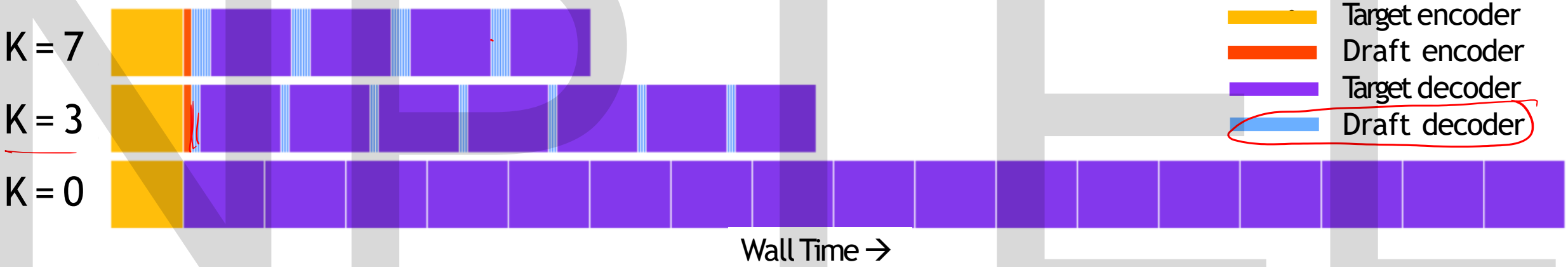
Content credits: https://youtu.be/S-8yr_RibJ4?si=Kv8xyyTsJvu8oKLV

Wall time speedup: Illustration on an encoder-decoder model



NPTEL

Wall time speedup: Illustration on an encoder-decoder model

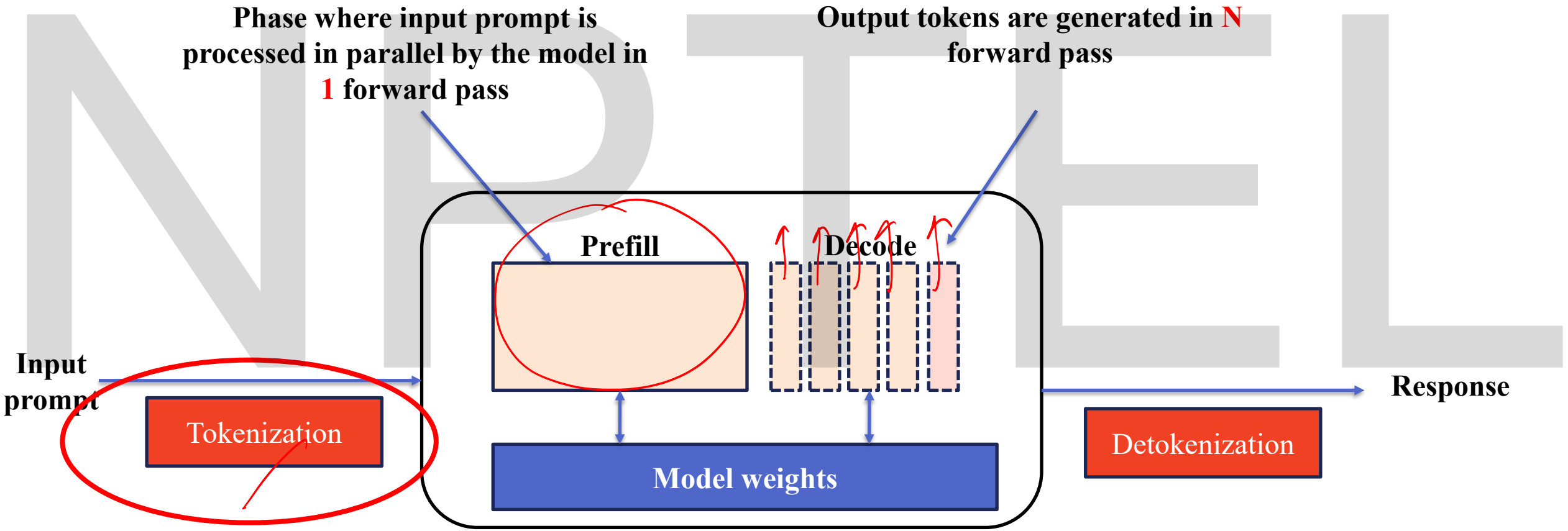


Agenda

- Speculative Decoding (contd.)
- **KV-caching**

NPTEL

Where can we improve?



Step 1a : Tokenization

NPTEL

Step 1a : Tokenization

token ~= 4 characters

– Different LLMs have Different Tokenizers

- Converting the input sentence/sequence into a sequence of LLM token ids

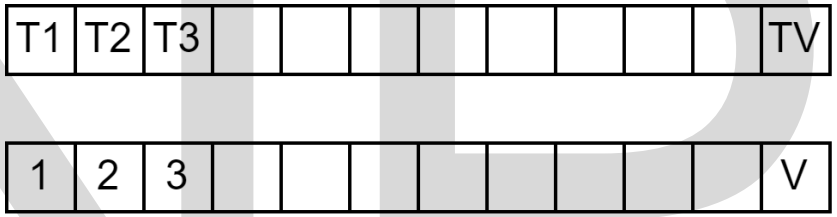
NPTEL

Step 1a : Tokenization

token ~= 4 characters

– Different LLMs have Different Tokenizers

- Converting the input sentence/sequence into a sequence of LLM token ids



Total vocabulary of tokens for the LLM

Llama – 125k volcab

Token ids corresponding to tokens

Convert input text /prompt → Tokenization → Vector of token ids

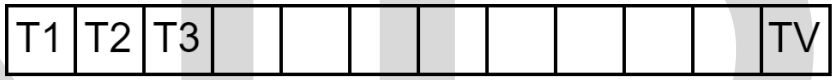
NPTEL

Step 1a : Tokenization

token ~= 4 characters

– Different LLMs have Different Tokenizers

- Converting the input sentence/sequence into a sequence of LLM token ids



Total vocabulary of tokens for the LLM Llama – 125k volcab



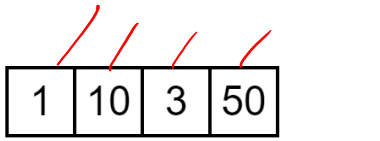
Token ids corresponding to tokens

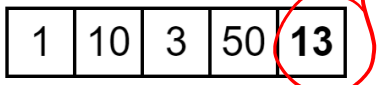
Convert input text /prompt → Tokenization → Vector of token ids

Make me sound smart

Make me sound smarter

Tokenization



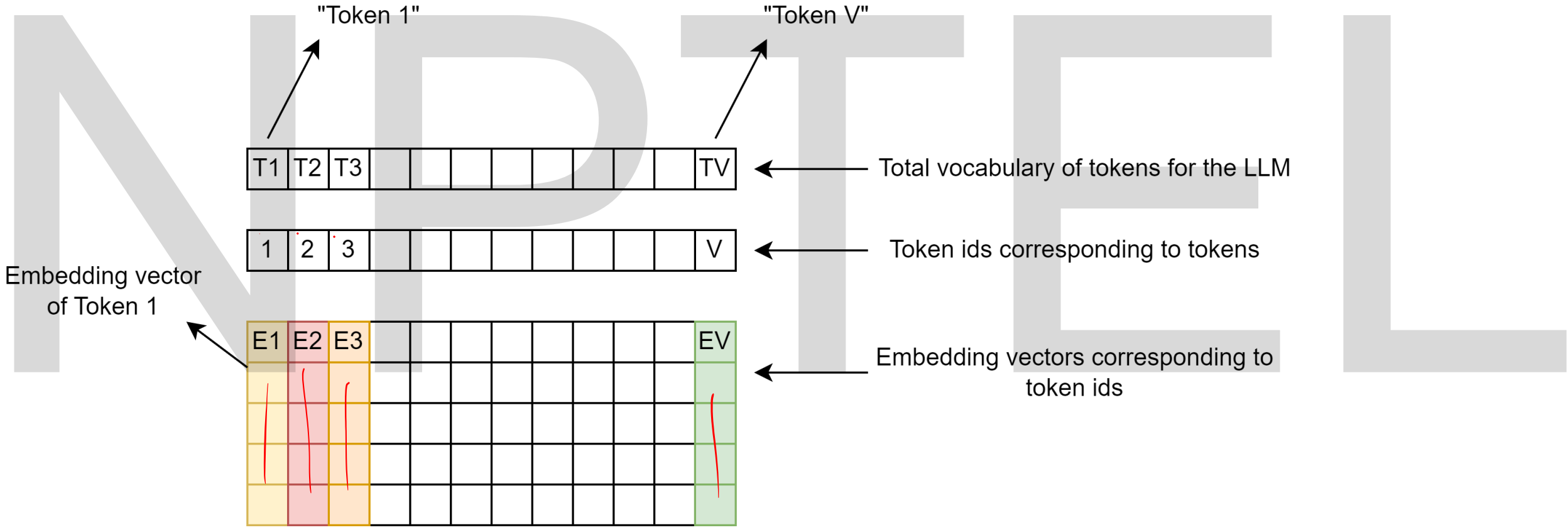


Vector representation of text

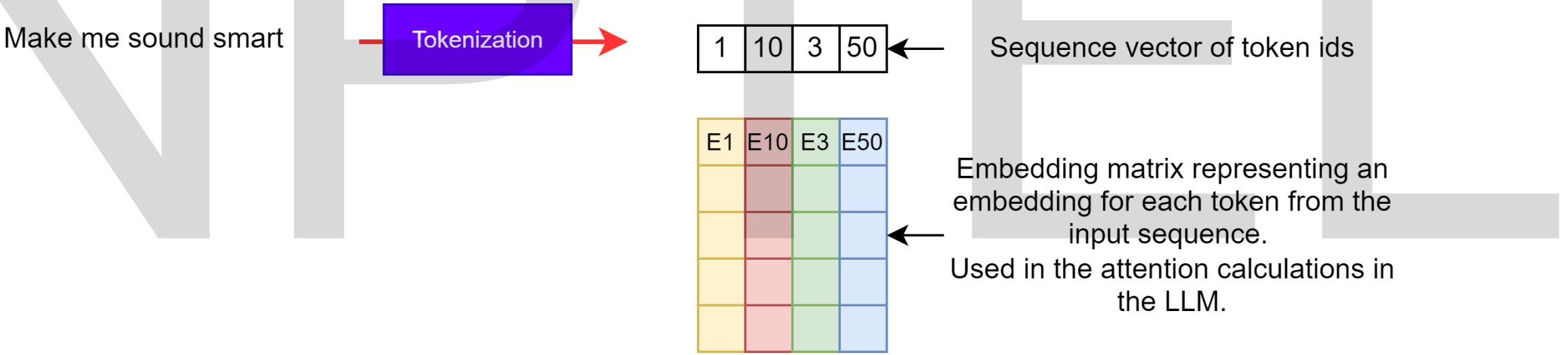
NPTEL

Step 1b. Convert each token to embedding vector

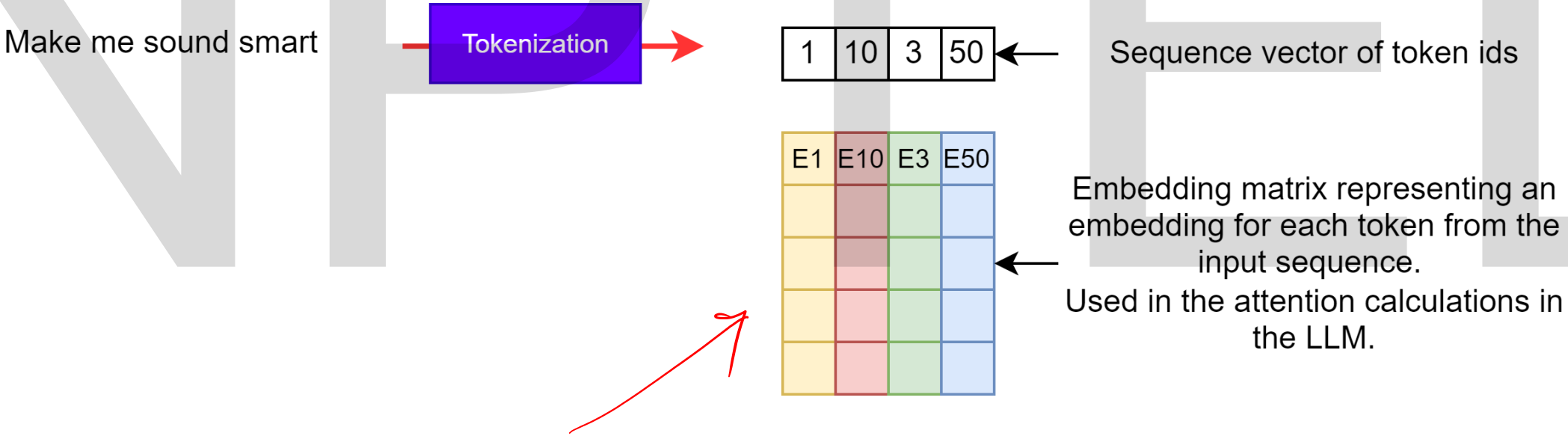
A sequence of human words is really a sequence of vectors aka a matrix



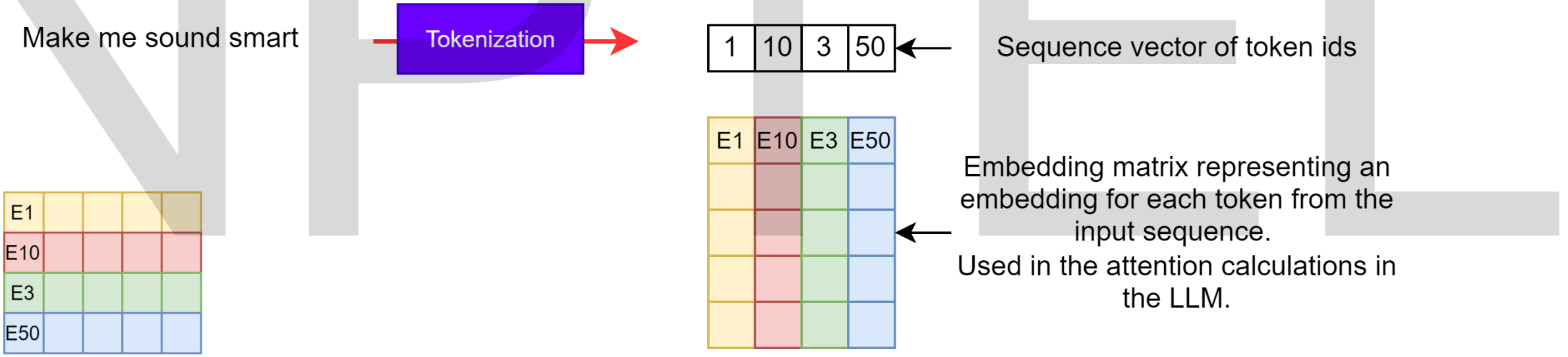
NPTEL


Make me sound smart
Tokenization
1
10
3
50
Sequence vector of token ids
E1
E10
E3
E50
Embedding matrix representing an embedding for each token from the input sequence.
Used in the attention calculations in the LLM.

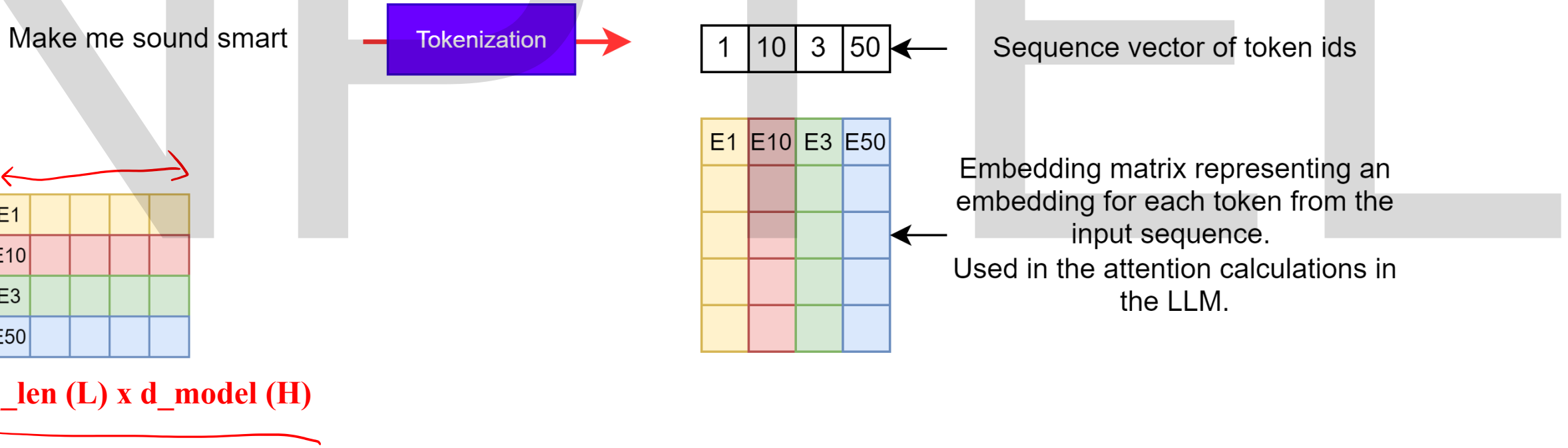
An input prompt is thus converted into a matrix ‘X’
-> Larger the prompt, larger the input matrix ‘X’



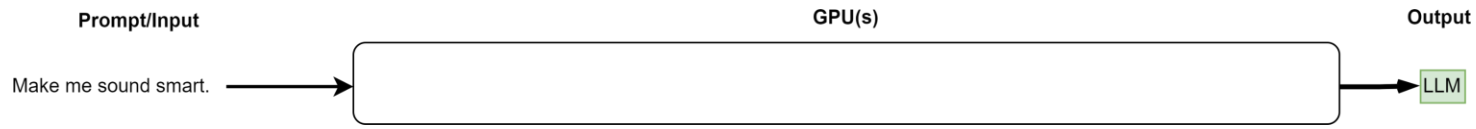
An input prompt is thus converted into a matrix ‘X’
-> Larger the prompt, larger the input matrix ‘X’



An input prompt is thus converted into a matrix ‘X’
-> Larger the prompt, larger the input matrix ‘X’



Step 2: Prefill







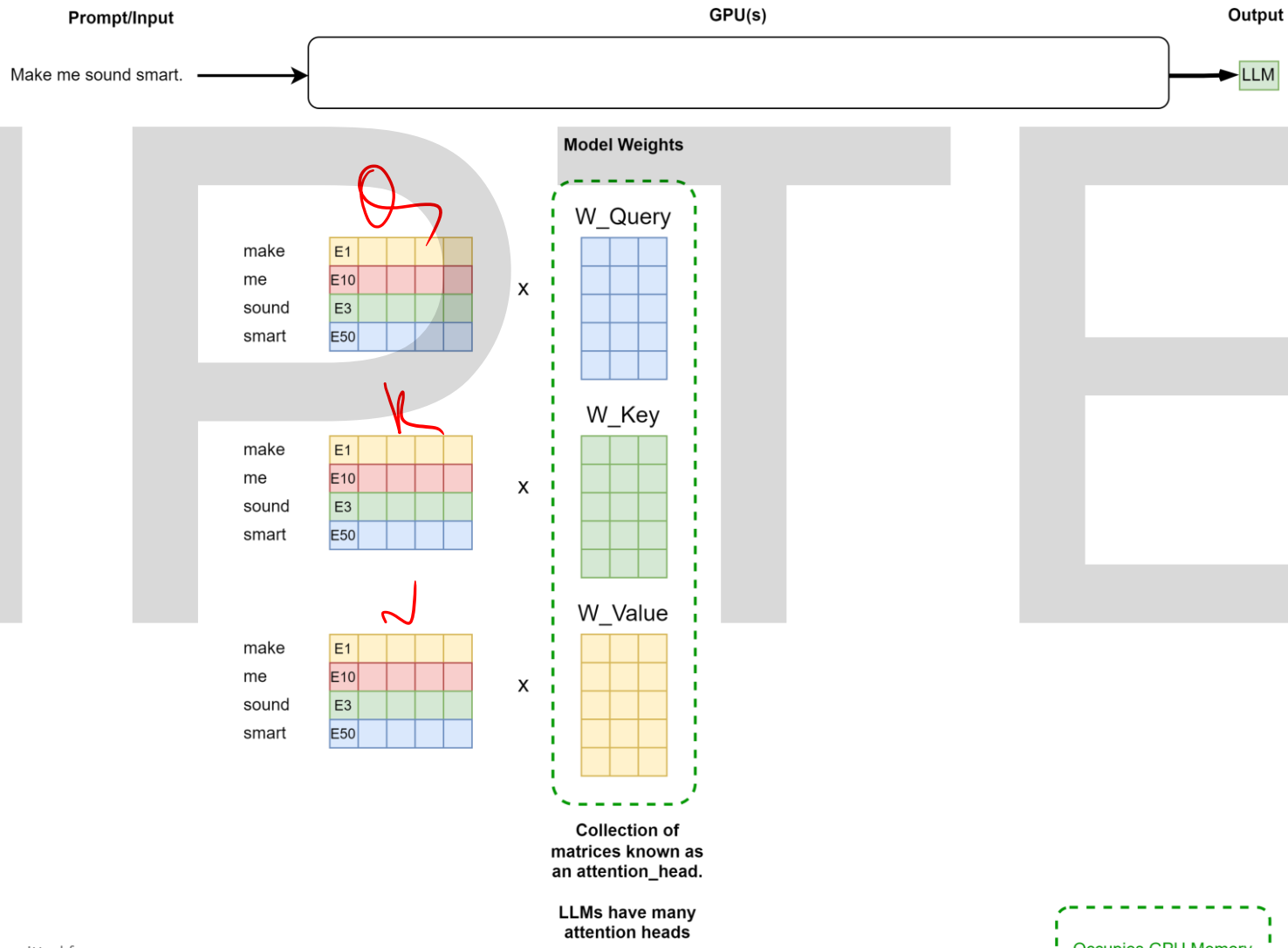


NPTEL

Many details of the model inference lifecycle are omitted for brevity such as mlps, layer-norms, decoding, etc.

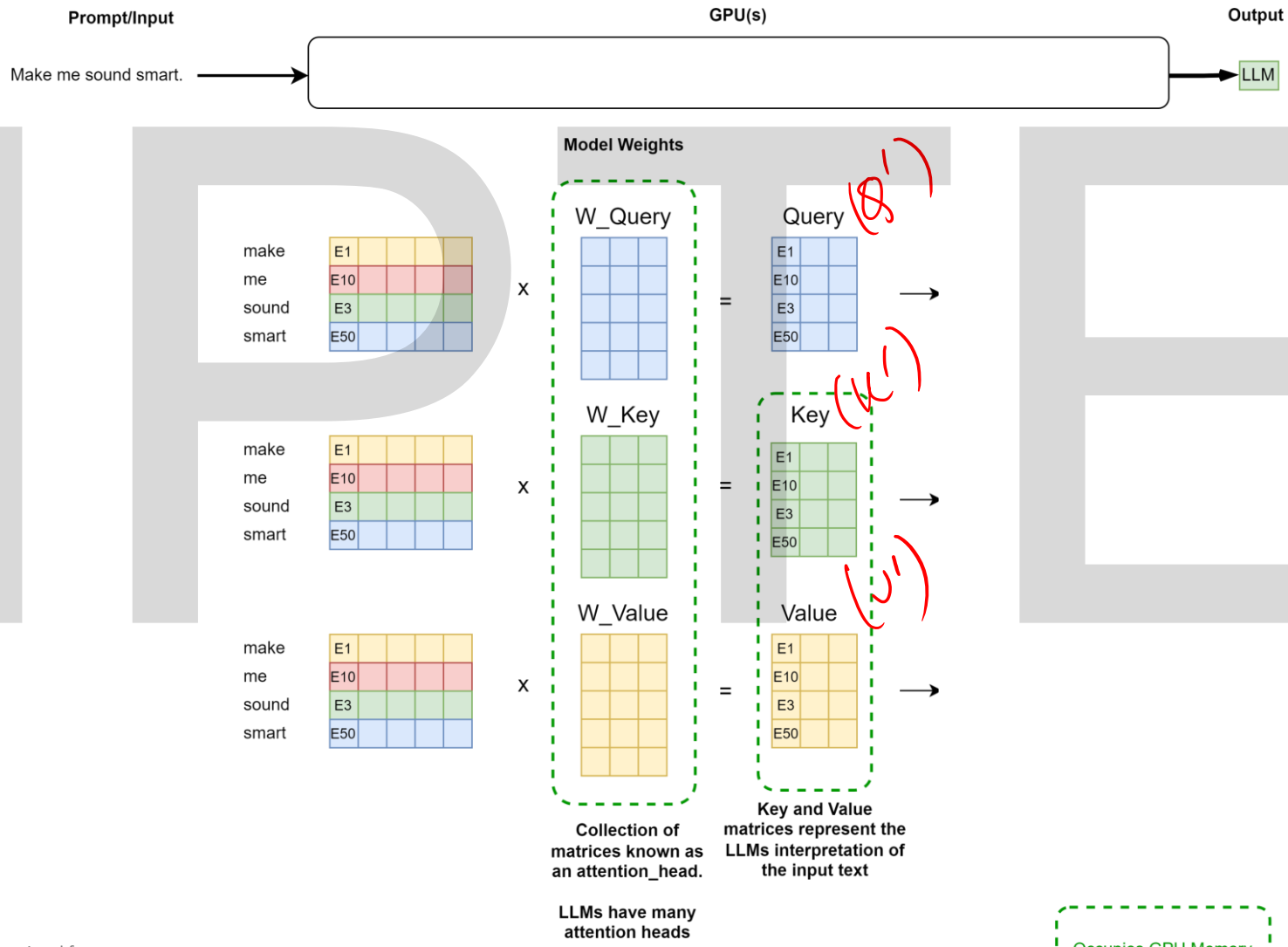
Occupies GPU Memory

Step 2: Prefill



Many details of the model inference lifecycle are omitted for brevity such as mlps, layer-norms, decoding, etc.

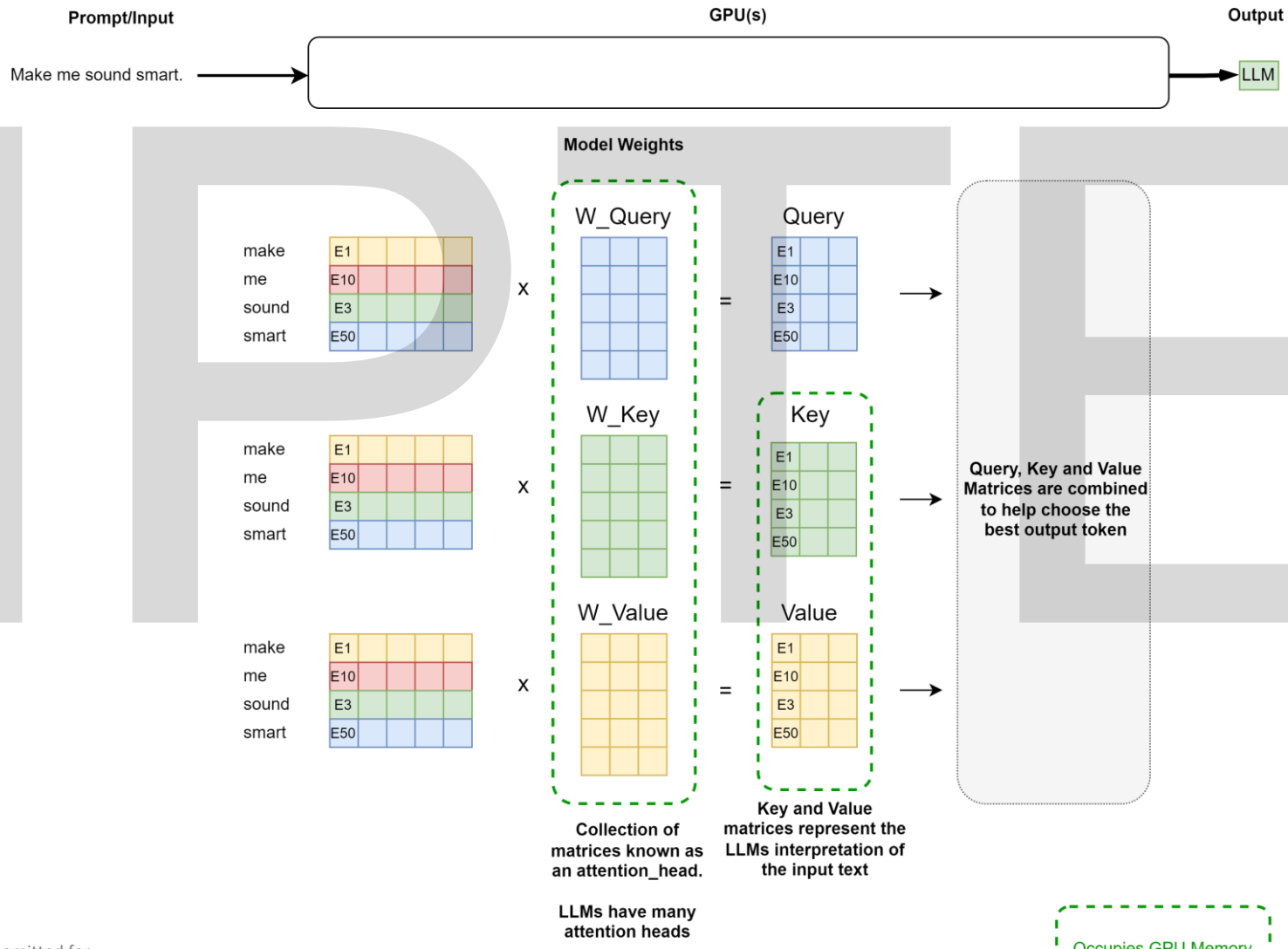
Step 2: Prefill



Occupies GPU Memory

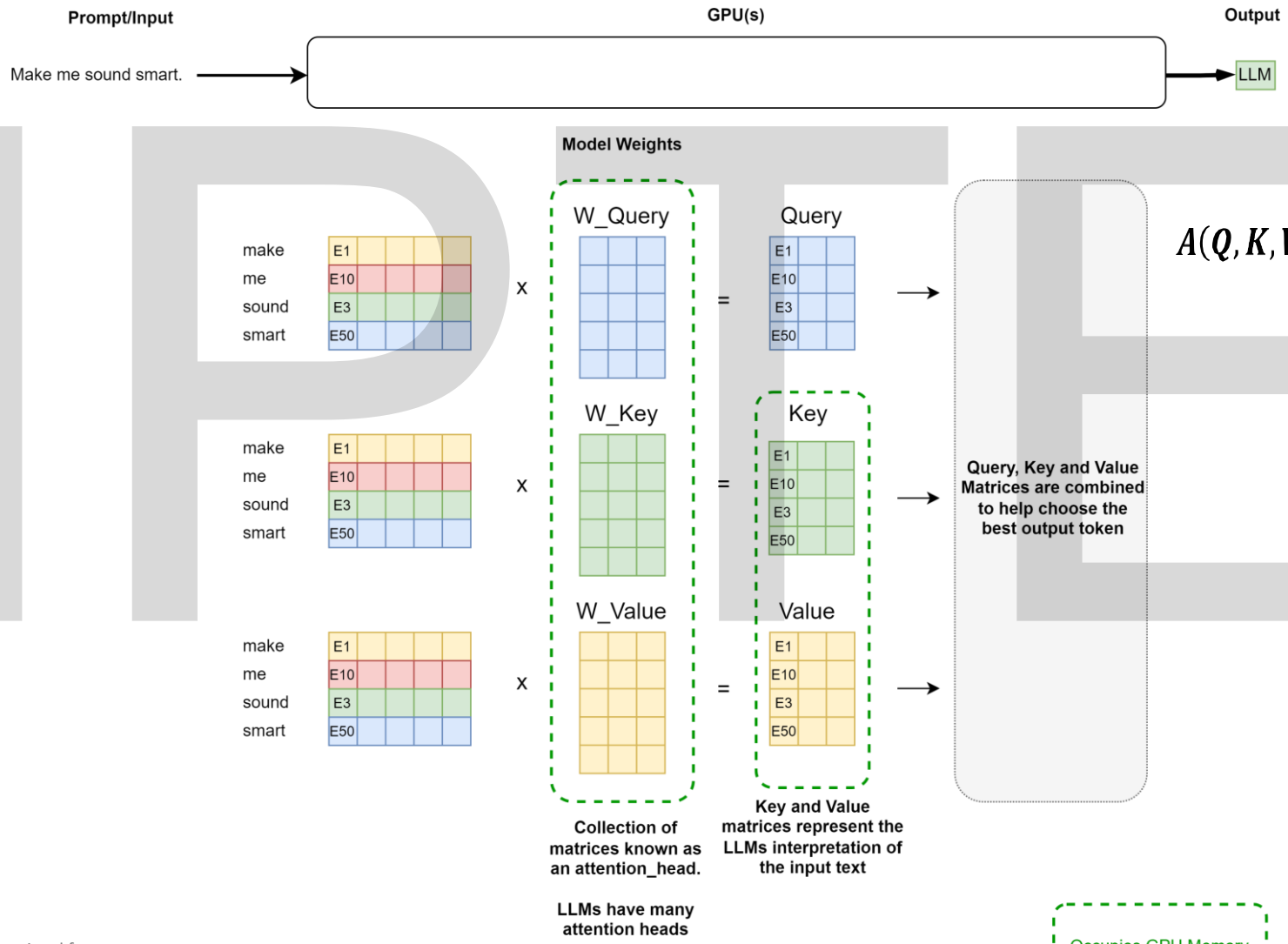
Many details of the model inference lifecycle are omitted for brevity such as mlps, layer-norms, decoding, etc.

Step 2: Prefill



Many details of the model inference lifecycle are omitted for brevity such as mlps, layer-norms, decoding, etc.

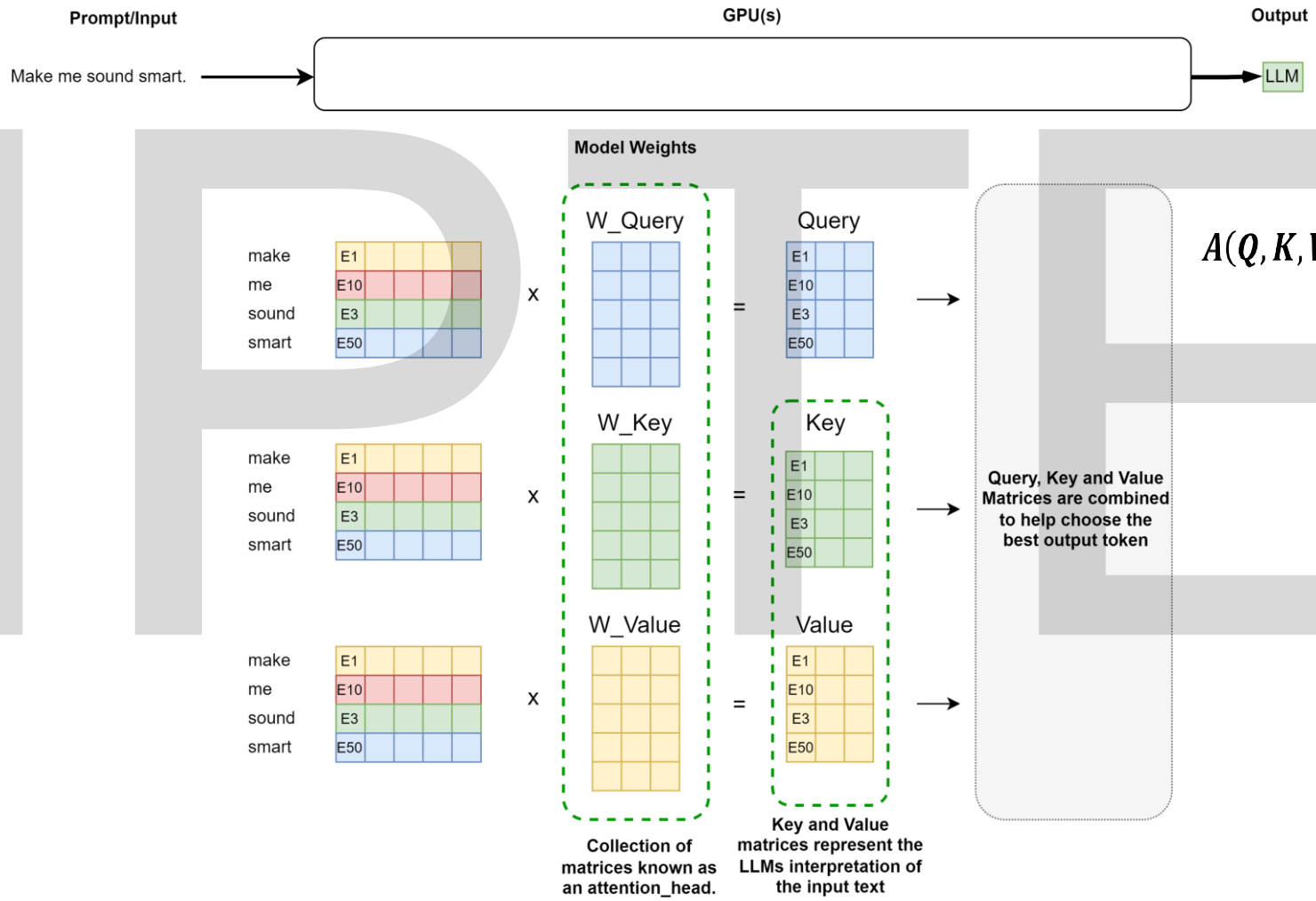
Step 2: Prefill



$$A(Q,K,V) = softmax\left(\frac{QK^T}{\sqrt{d}}\right)V$$

Many details of the model inference lifecycle are omitted for brevity such as mlps, layer-norms, decoding, etc.

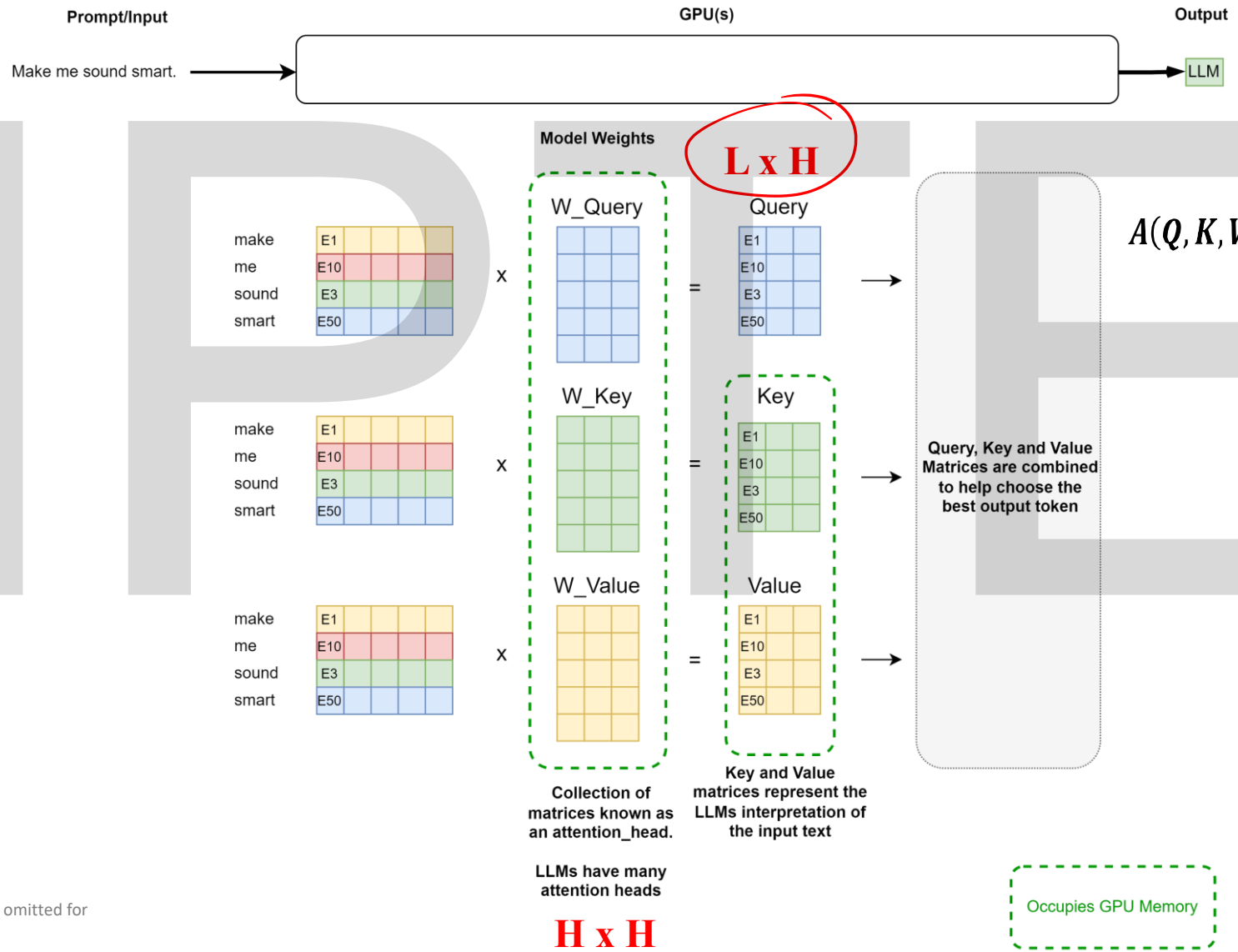
Step 2: Prefill



$$A(Q,K,V) = softmax\left(\frac{QK^T}{\sqrt{d}}\right)V$$

Many details of the model inference lifecycle are omitted for brevity such as mlps, layer-norms, decoding, etc.

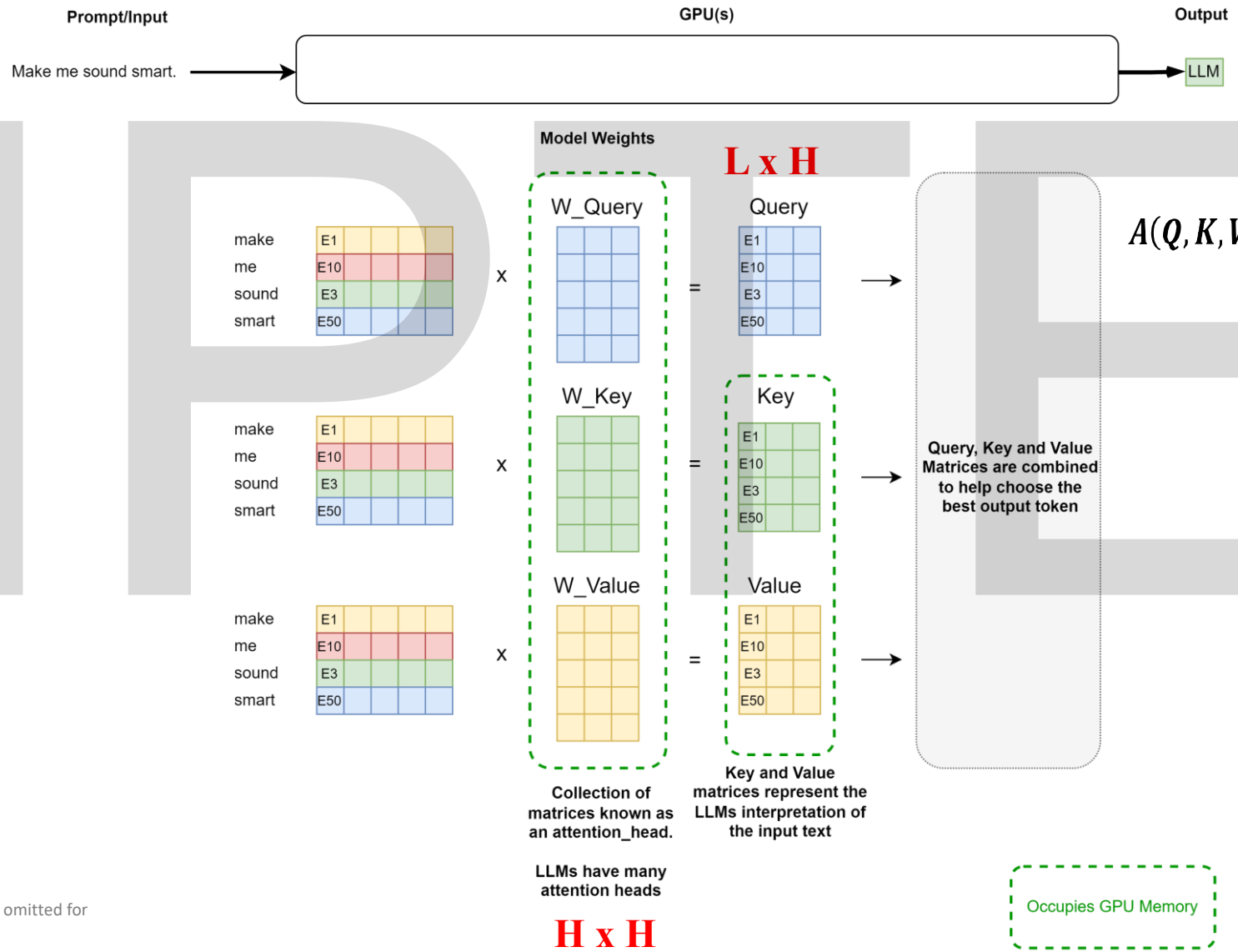
Step 2: Prefill



$$A(Q,K,V) = softmax\left(\frac{QK^T}{\sqrt{d}}\right)V$$

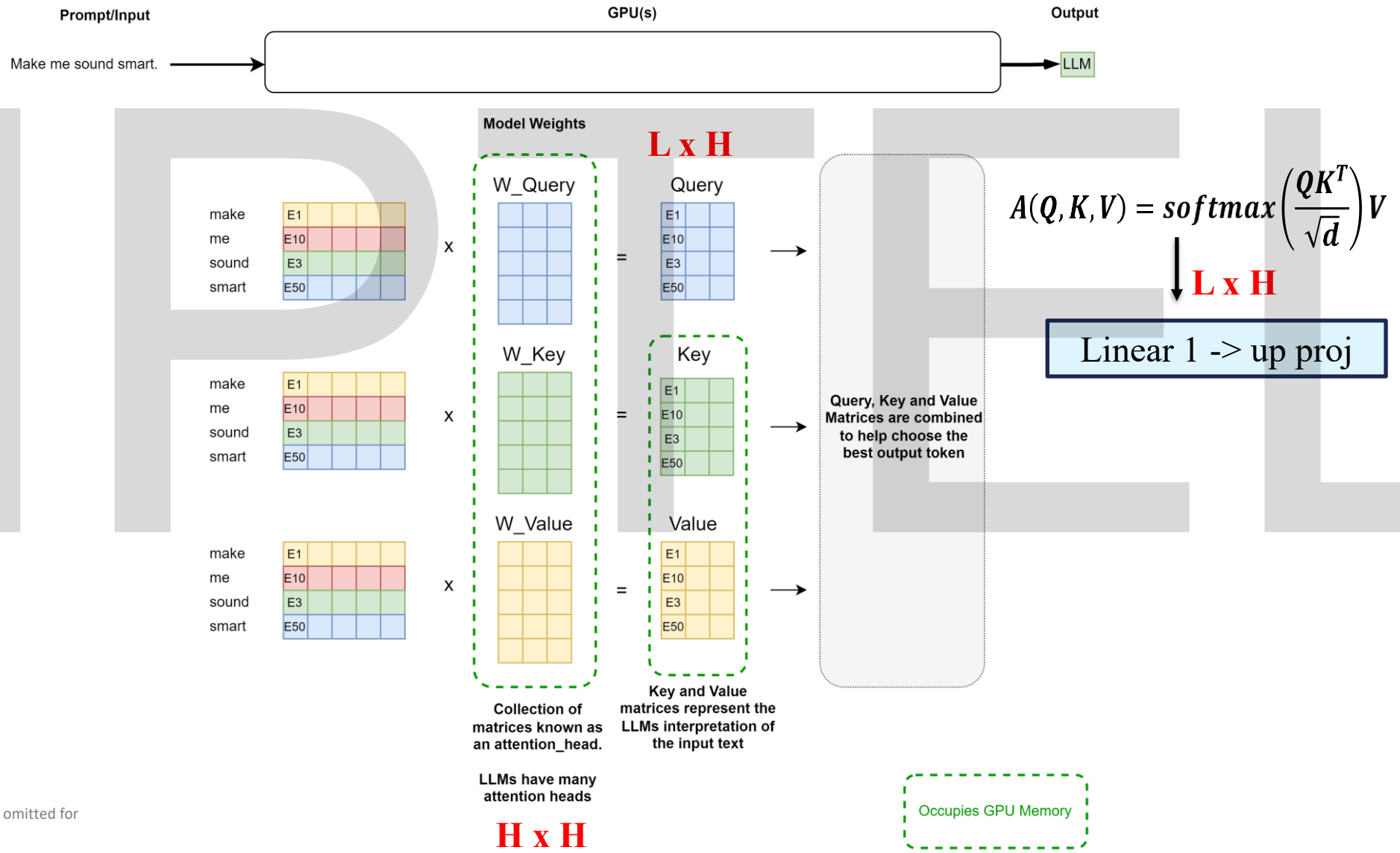
Many details of the model inference lifecycle are omitted for brevity such as mlps, layer-norms, decoding, etc.

Step 2: Prefill



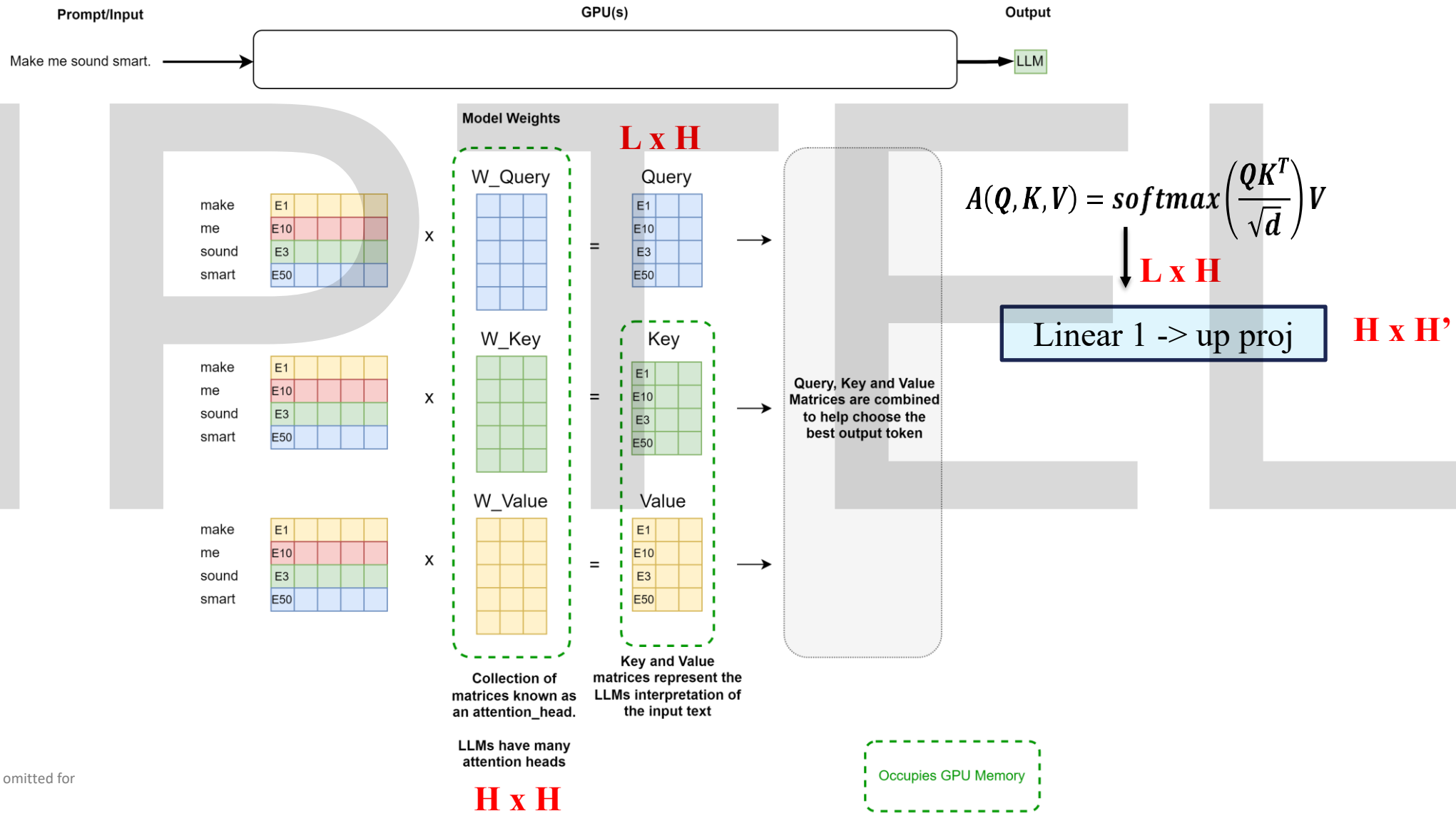
Many details of the model inference lifecycle are omitted for brevity such as mlps, layer-norms, decoding, etc.

Step 2: Prefill



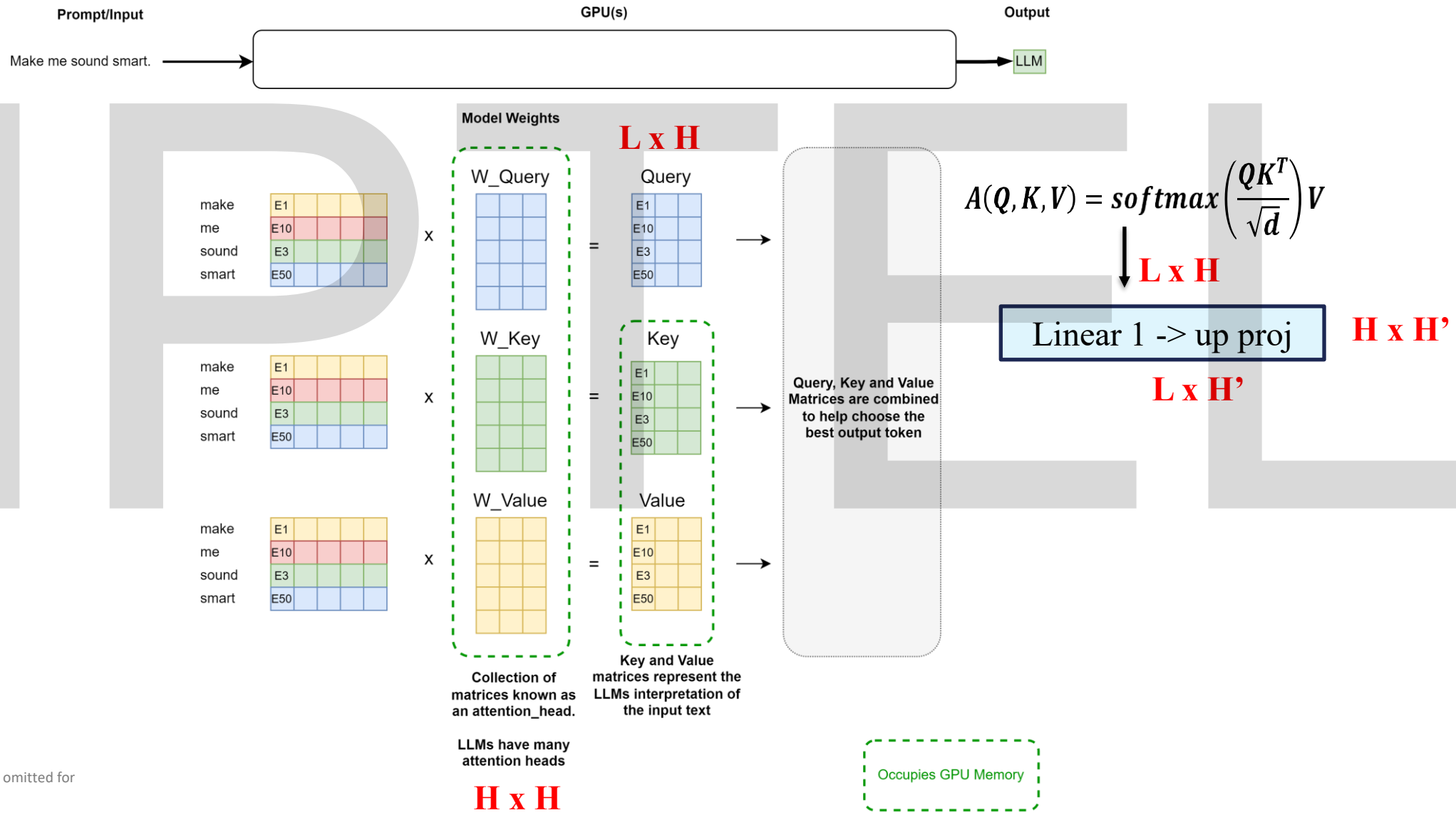
Many details of the model inference lifecycle are omitted for brevity such as mlps, layer-norms, decoding, etc.

Step 2: Prefill



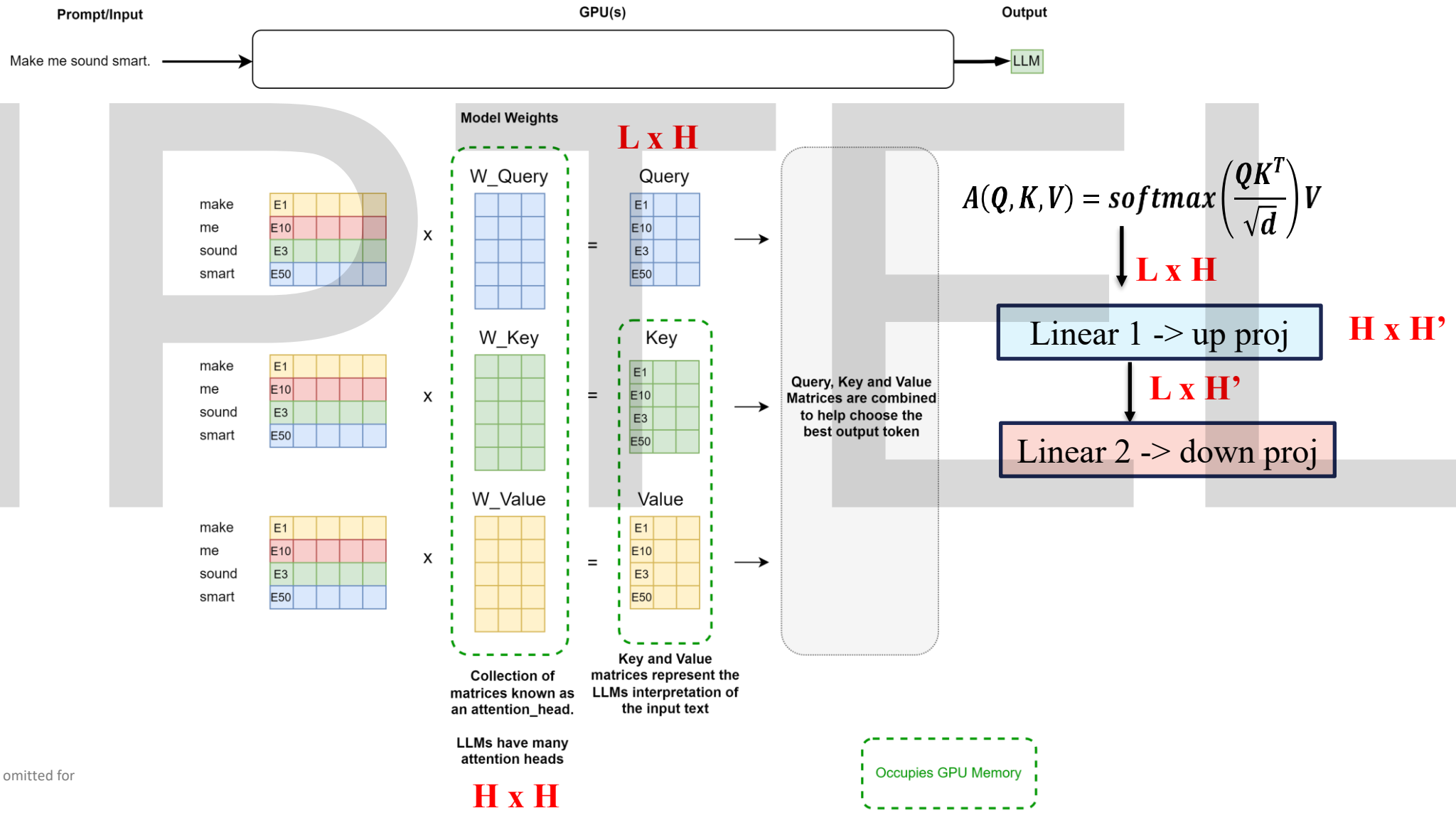
Many details of the model inference lifecycle are omitted for brevity such as mlps, layer-norms, decoding, etc.

Step 2: Prefill



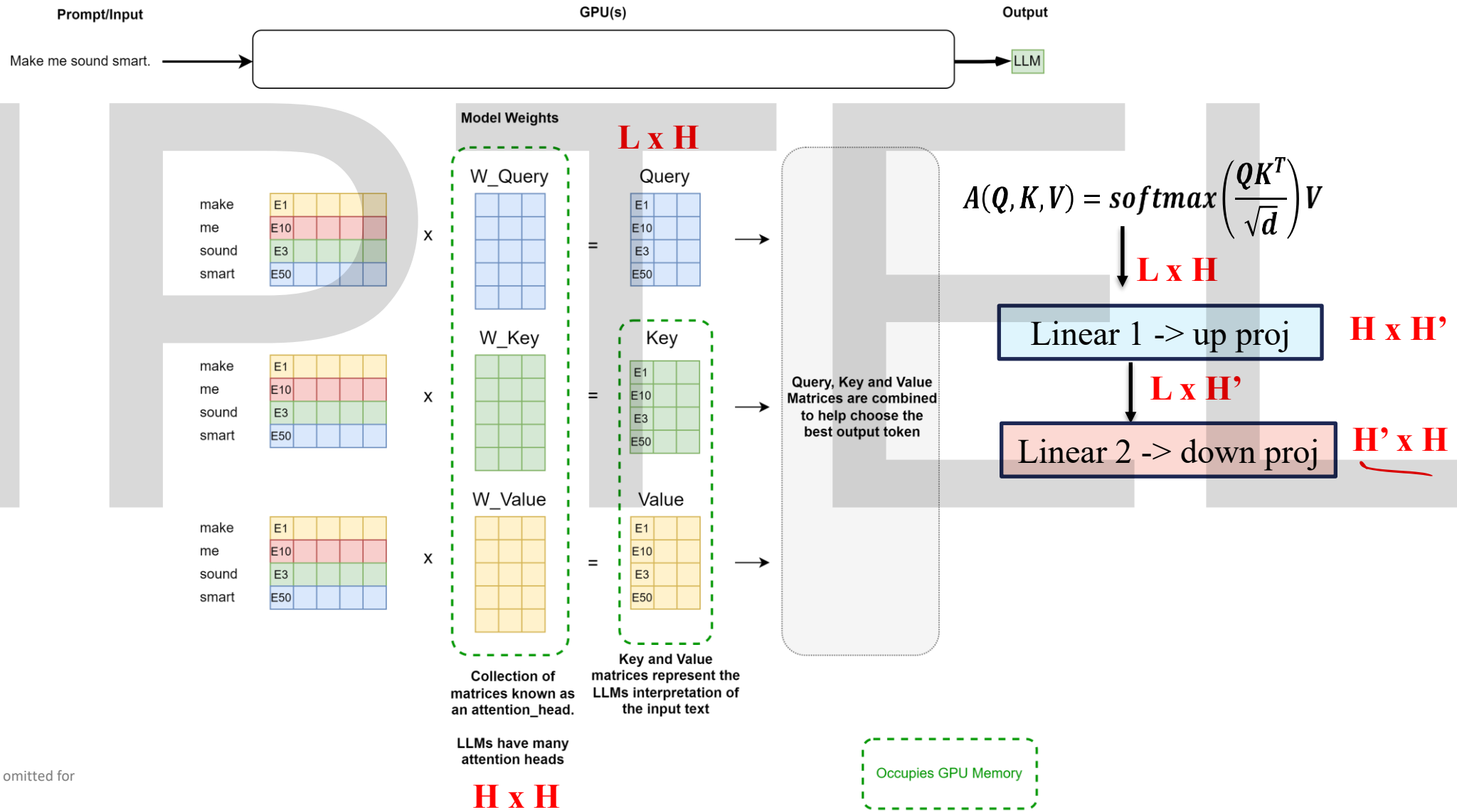
Many details of the model inference lifecycle are omitted for brevity such as mlps, layer-norms, decoding, etc.

Step 2: Prefill



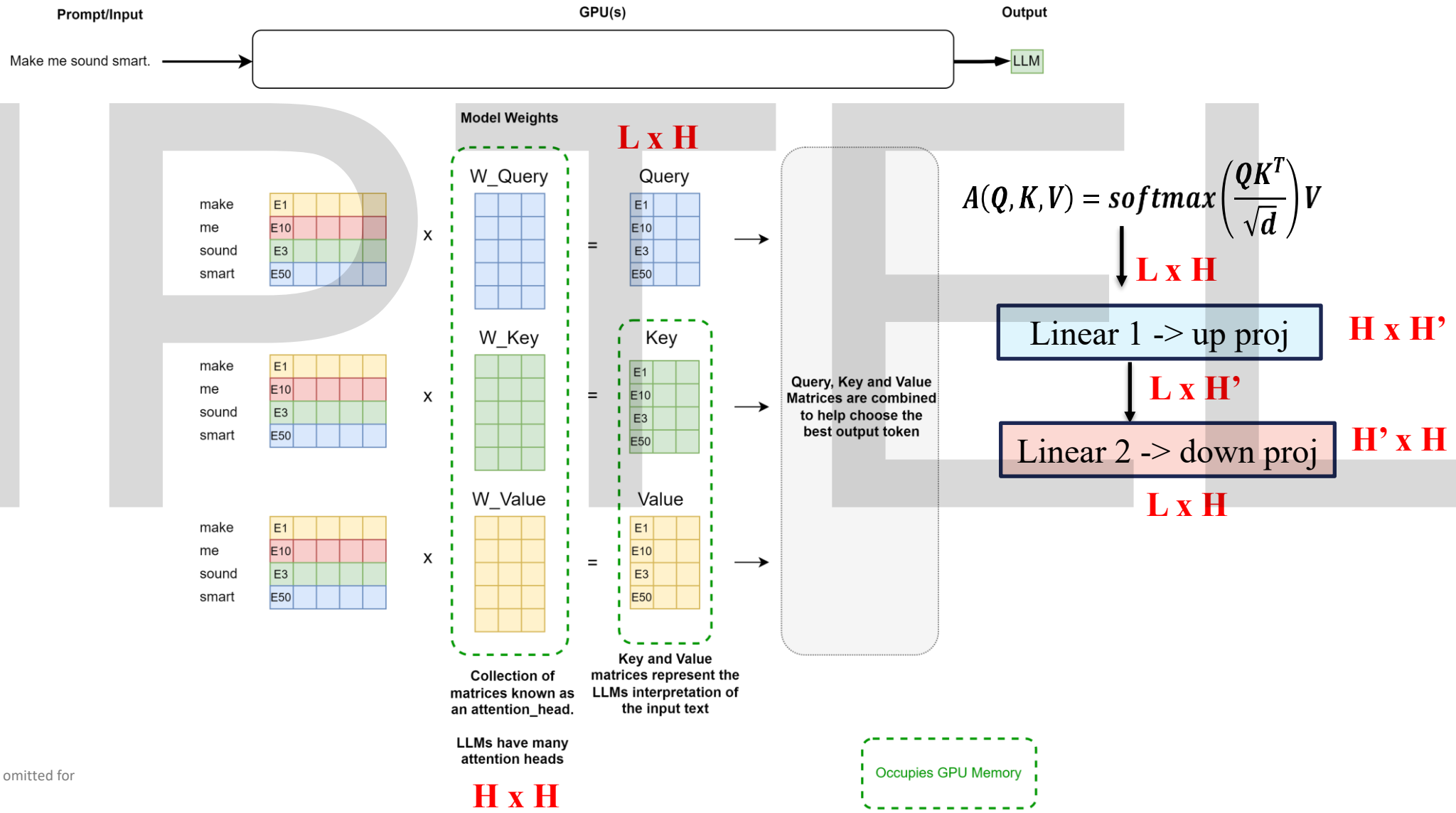
Many details of the model inference lifecycle are omitted for brevity such as mlps, layer-norms, decoding, etc.

Step 2: Prefill



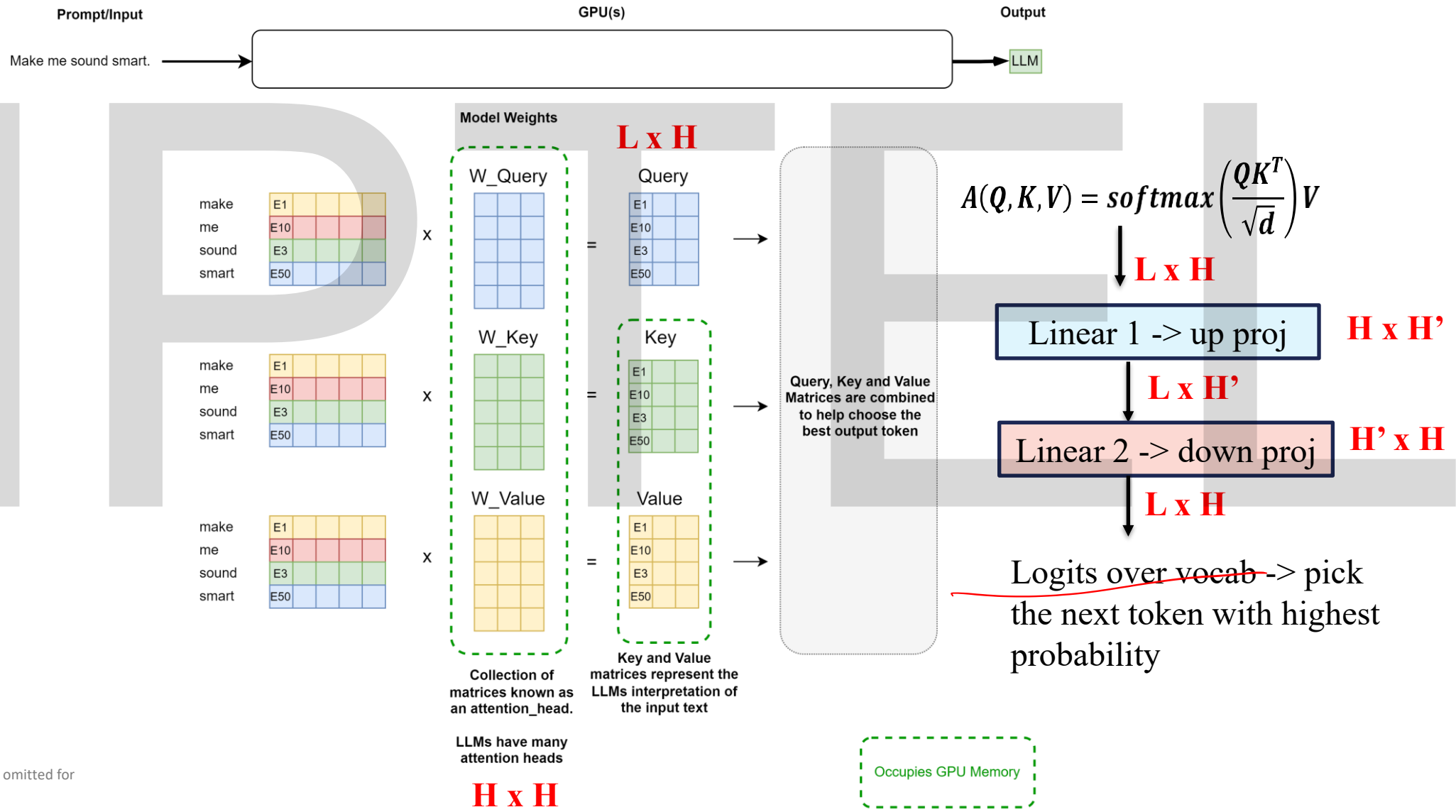
Many details of the model inference lifecycle are omitted for brevity such as mlps, layer-norms, decoding, etc.

Step 2: Prefill



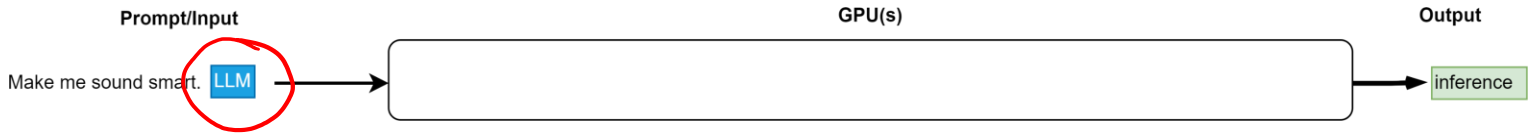
Many details of the model inference lifecycle are omitted for brevity such as mlps, layer-norms, decoding, etc.

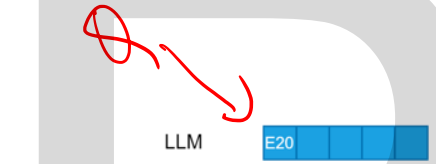
Step 2: Prefill

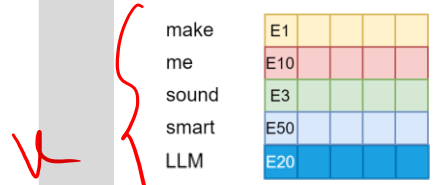


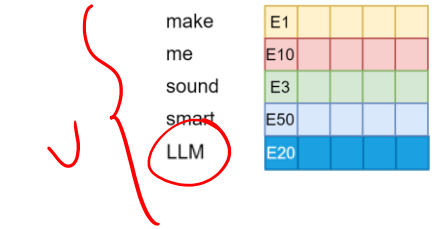
Many details of the model inference lifecycle are omitted for brevity such as mlps, layer-norms, decoding, etc.

Step 3 : Decode(s)







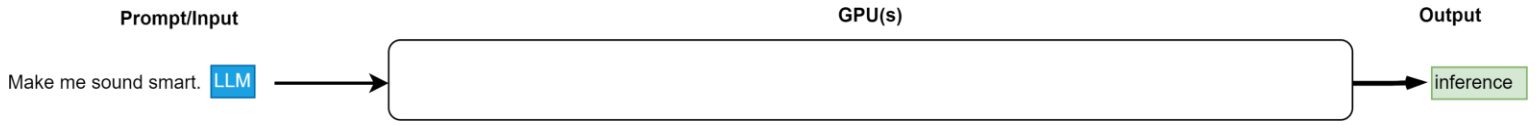


Query, Key and Value Matrices are combined to help choose the best output token

LLMs have many attention heads

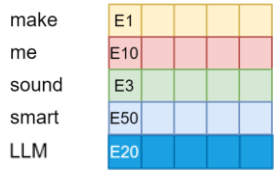
NPTEL

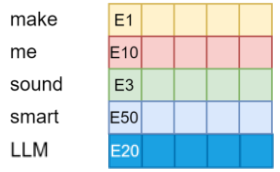
Step 3 : Decode(s)





(L+1) x H



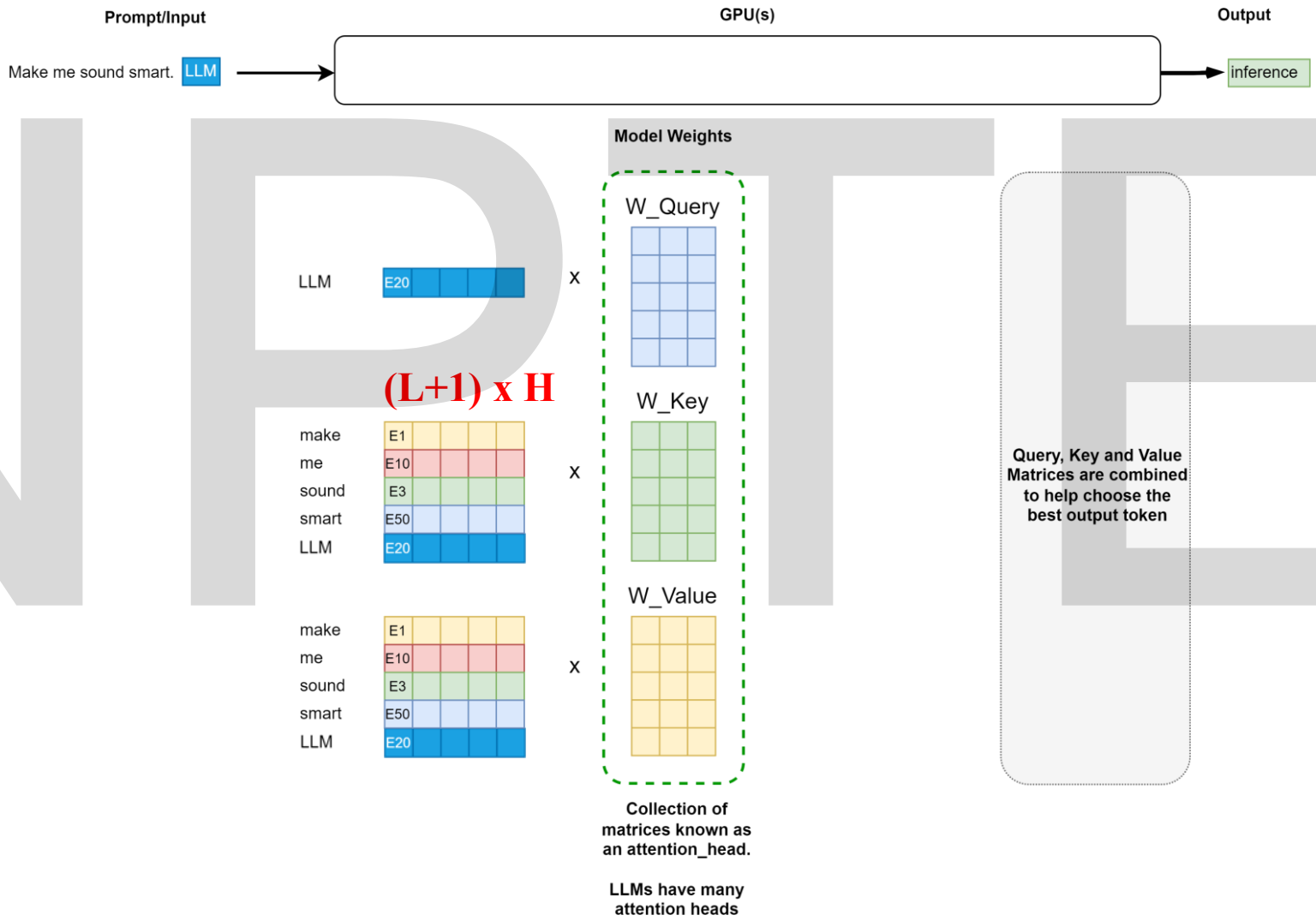


Query, Key and Value Matrices are combined to help choose the best output token

LLMs have many attention heads

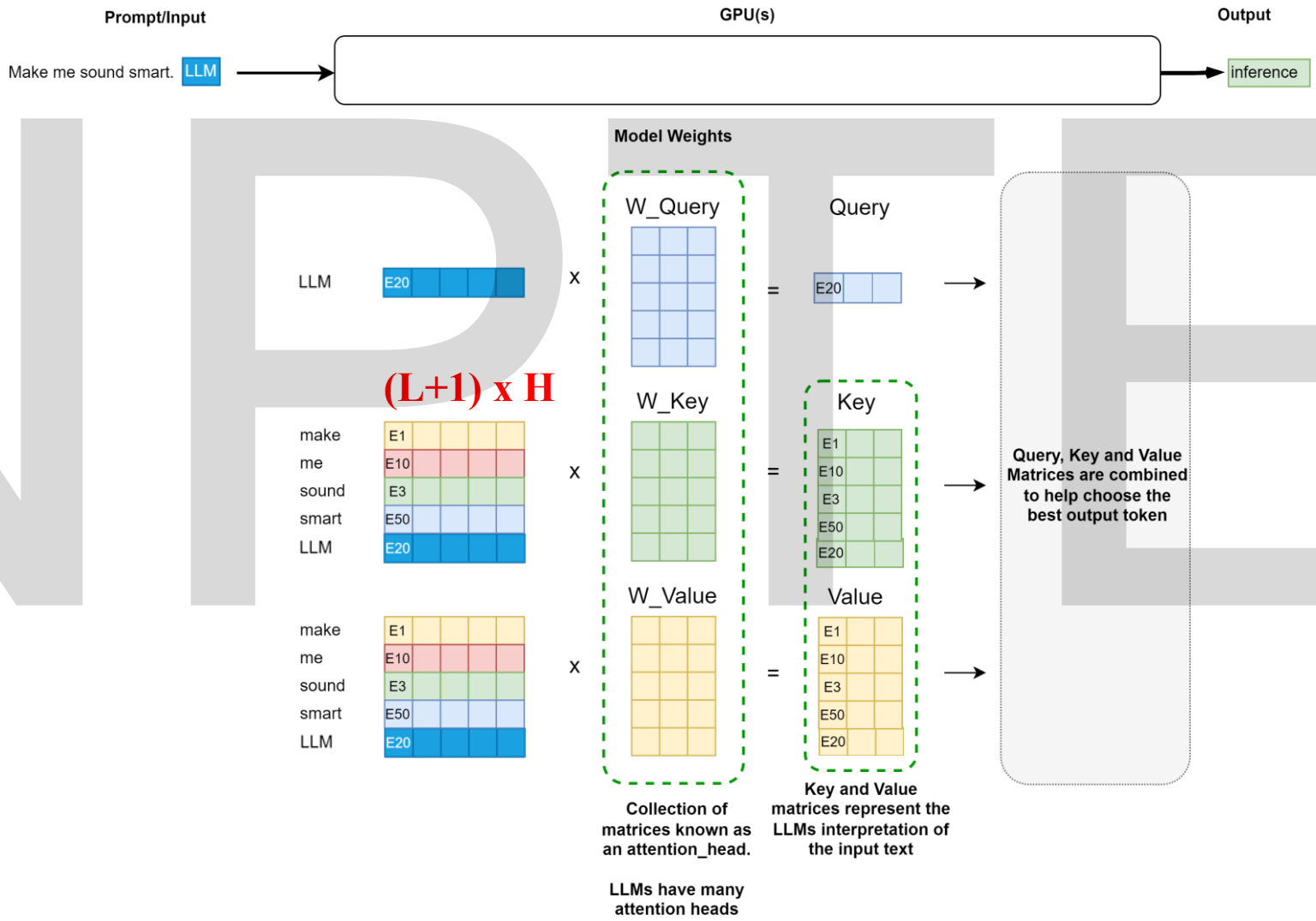
NPTEL

Step 3 : Decode(s)

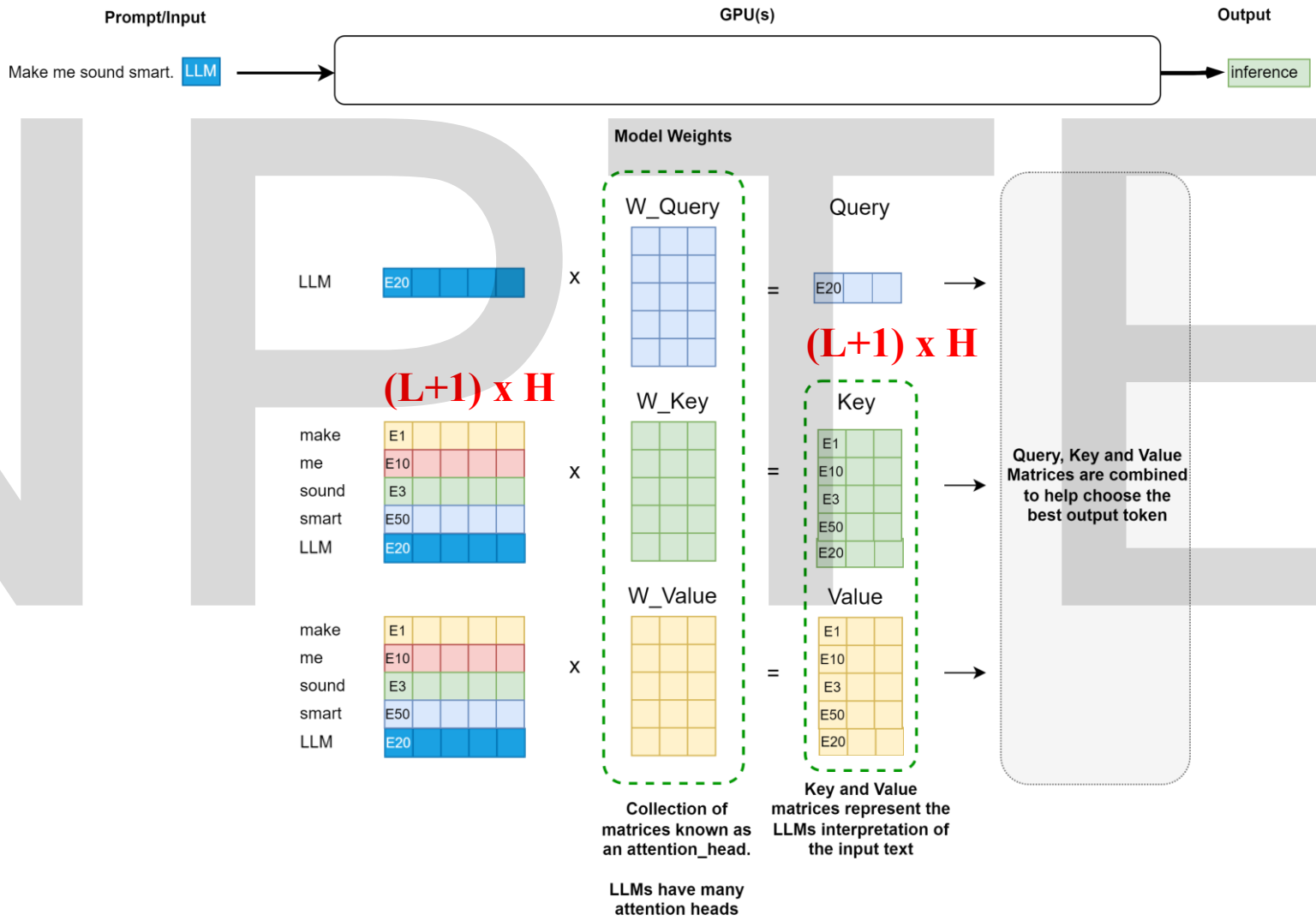


NPTEL

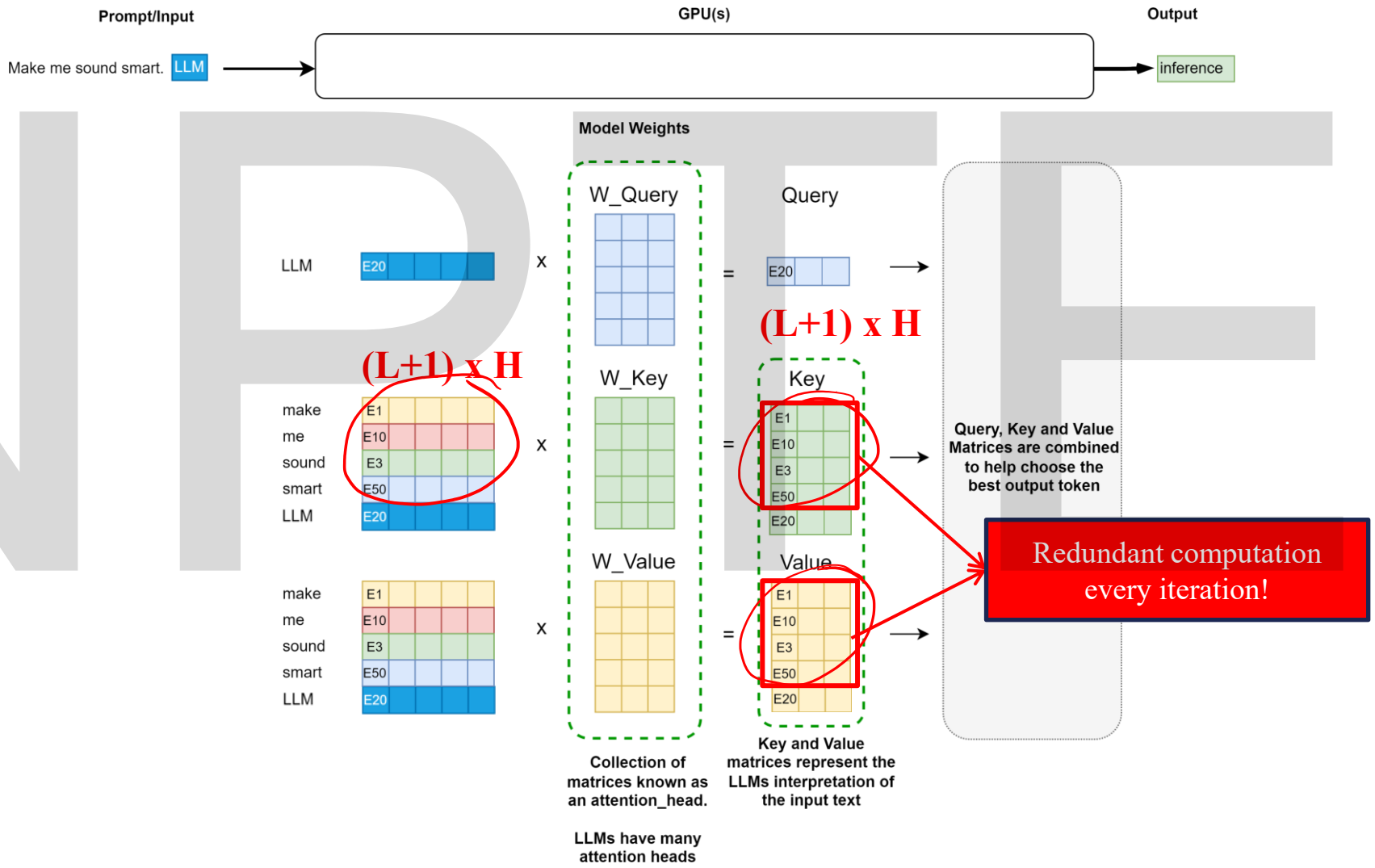
Step 3 : Decode(s)



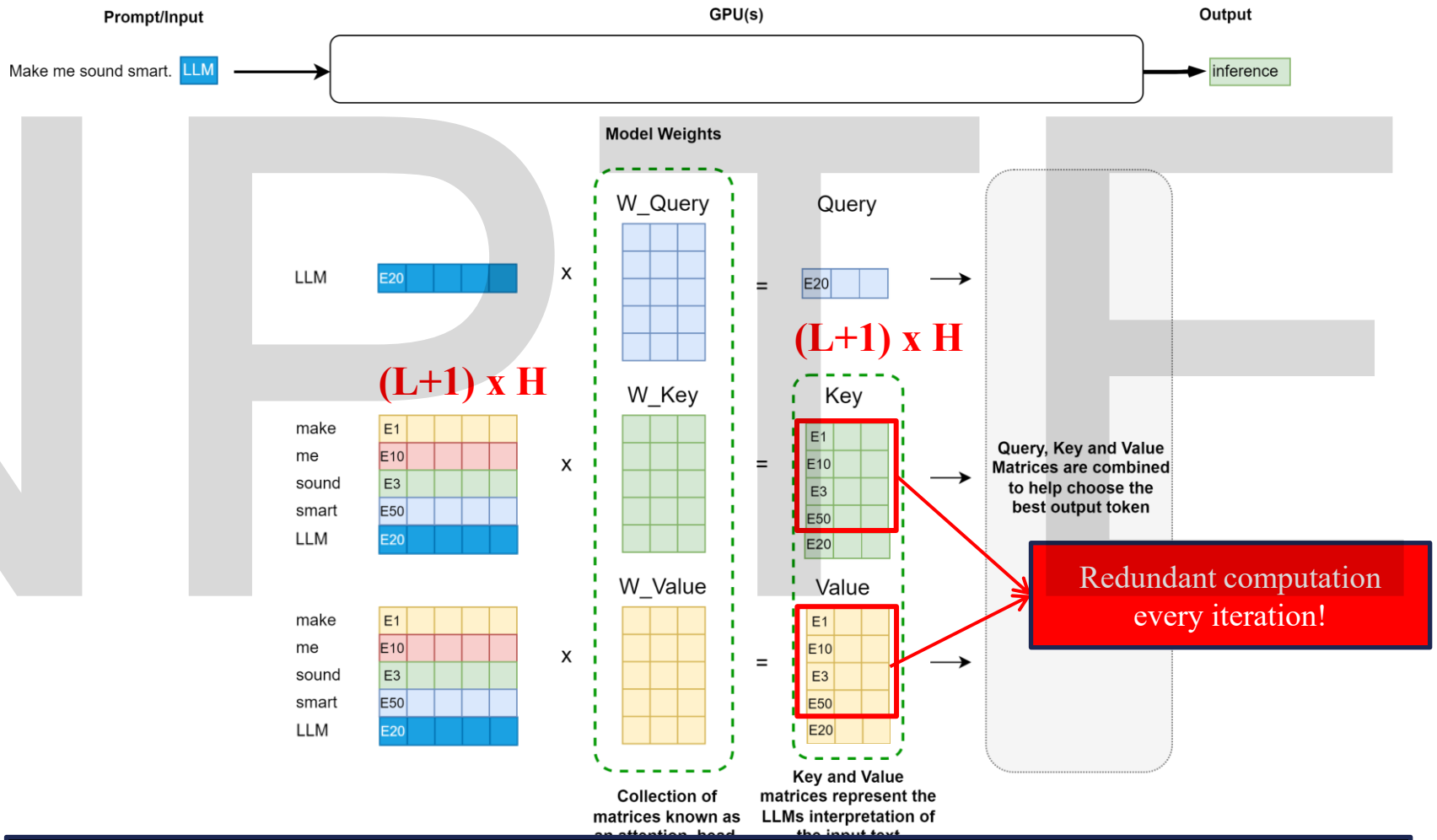
Step 3 : Decode(s)



Step 3 : Decode(s)

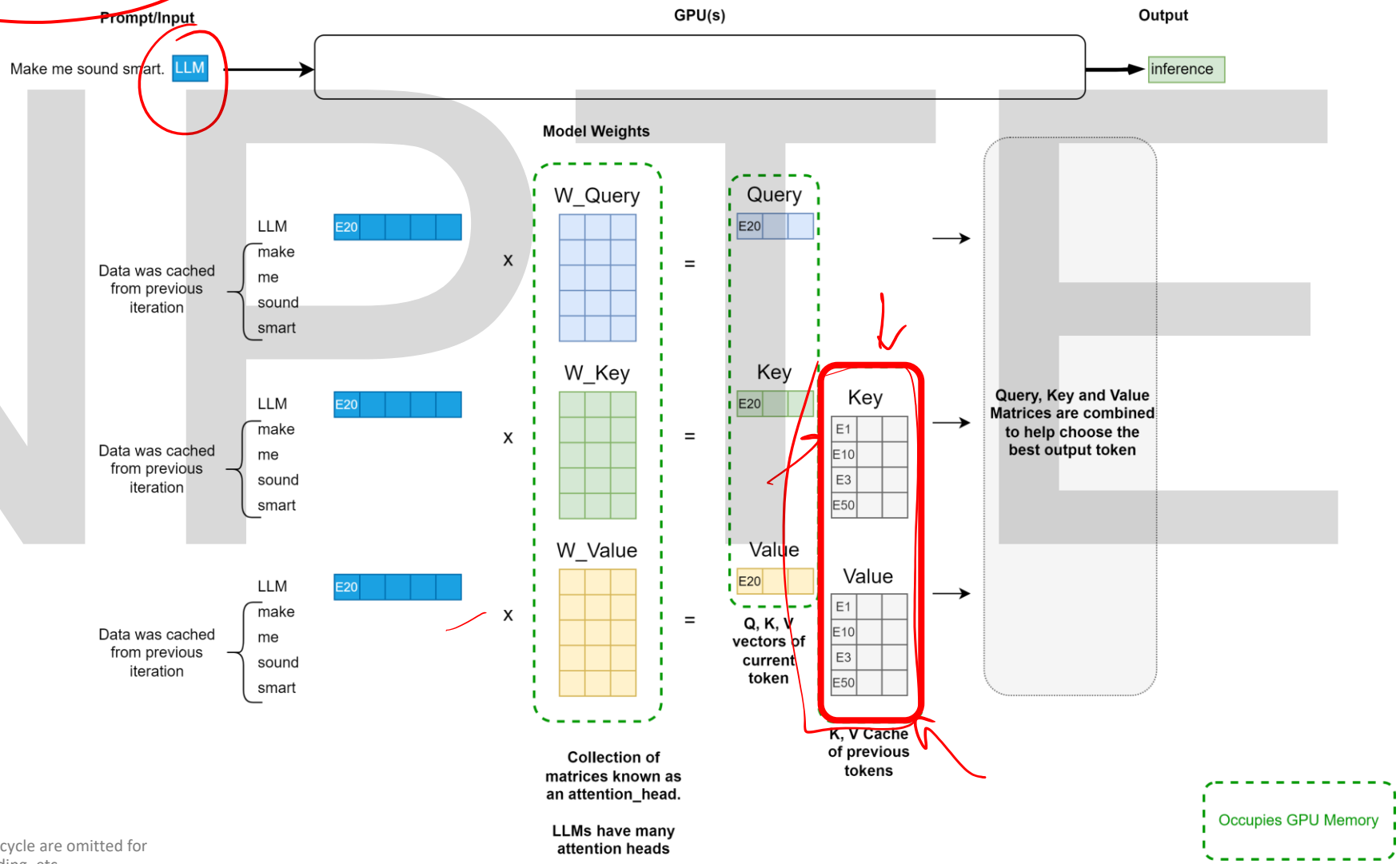


Step 3 : Decode(s)



Can we speed this up somehow?

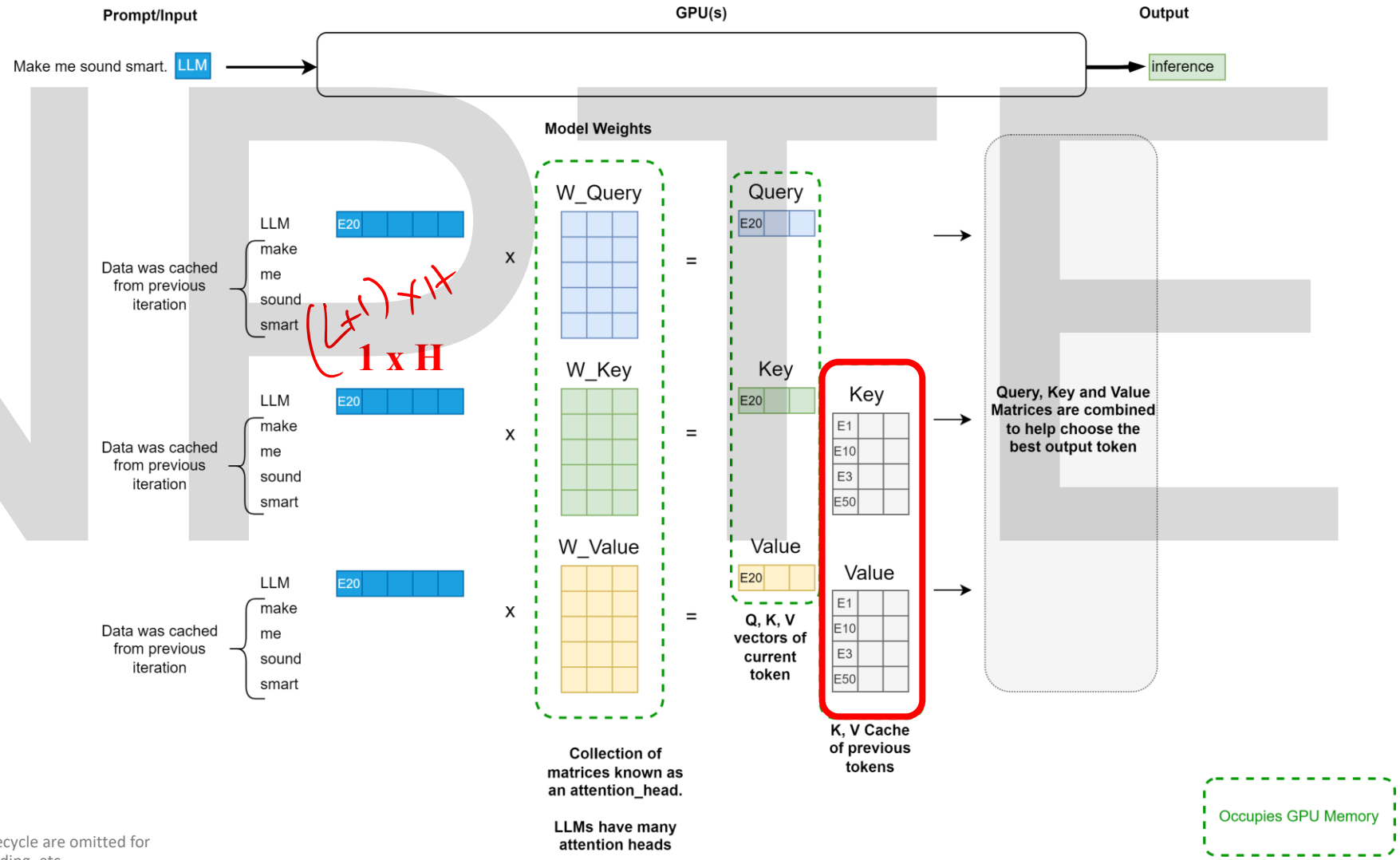
KV cache to the rescue!



Occupies GPU Memory

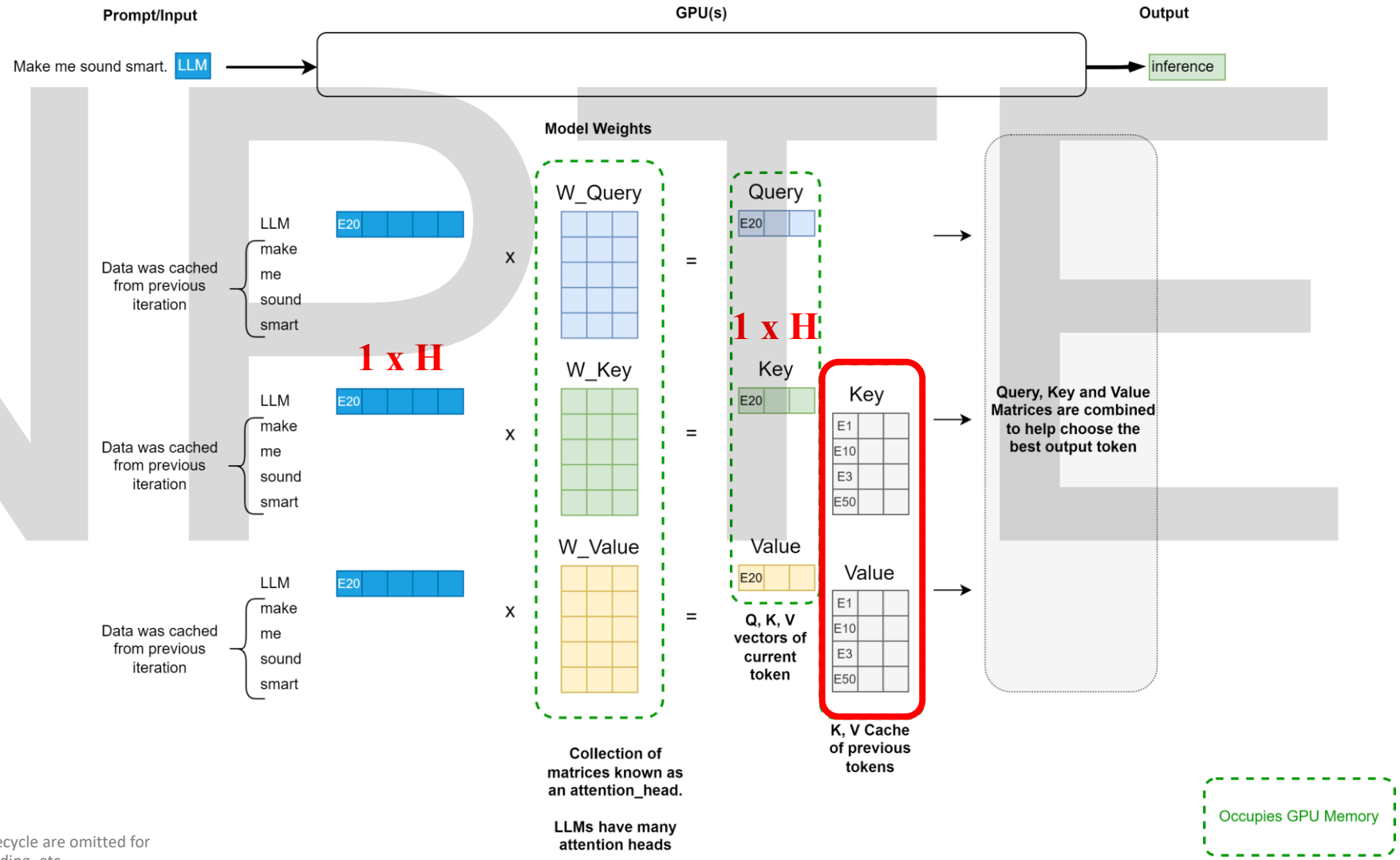
Many details of the model inference lifecycle are omitted for brevity such as mlps, layer-norms, decoding, etc

KV cache to the rescue!



Many details of the model inference lifecycle are omitted for brevity such as mlps, layer-norms, decoding, etc

KV cache to the rescue!



Many details of the model inference lifecycle are omitted for brevity such as mlps, layer-norms, decoding, etc



THANK YOU

Slide Courtesy: Murali Annavaram (University of Southern California), R. Govindarajan (IISc), Arkaprava Basu (IISc), Jayashree Mohan (Microsoft Research)